

LAPORAN TUGAS BESAR 1 IF3270

PEMBELAJARAN MESIN

Feed Forward Neural Network



Dosen Pengampu : Dr. Nur Ulfa Maulidevi, S.T, M.Sc.

Disusun oleh:

Aurelius Justin Philo Fanjaya (13522020)

Amalia Putri (13522042)

Fedrianz Dharma (13522090)

**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

2025

DAFTAR ISI

DAFTAR ISI.....	2
DAFTAR GAMBAR.....	2
DAFTAR TABEL.....	3
1. Deskripsi Persoalan.....	5
2. Pembahasan.....	5
2.1 Penjelasan implementasi.....	5
2.1.1 Deskripsi kelas beserta deskripsi atribut dan methodnya.....	5
2.1.2 Penjelasan forward propagation.....	13
2.1.3 Penjelasan Backward Propagation dan Weight Update.....	14
2.2 Hasil pengujian.....	17
2.2.1 Pengaruh depth dan width.....	17
2.2.2 Pengaruh fungsi aktivasi.....	30
2.2.3 Pengaruh learning rate.....	36
2.2.4 Pengaruh inisialisasi bobot.....	45
2.2.5 Pengaruh regularisasi.....	53
2.2.6 Perbandingan dengan library sklearn.....	59
3. Kesimpulan dan Saran.....	62
3.1 Kesimpulan.....	62
3.2 Saran.....	62
4. Pembagian tugas tiap anggota kelompok.....	63
5. Referensi.....	63

DAFTAR GAMBAR

Gambar 2.1.2.1 Total Loss.....	13
Gambar 2.1.2.2 Method Call Kelas FFNN.....	13
Gambar 2.1.2.2 Method Call Kelas Layer.....	14
Gambar 2.1.2.3 Method Call Kelas Neuron.....	14
Gambar 2.1.3.1 Weight Update.....	15
Gambar 2.1.3.2 Method Backward Kelas Layer.....	15
Gambar 2.1.3.2 Method Backward Kelas Neuron.....	16
Gambar 2.1.3.3 Update Kelas Layer.....	16
Gambar 2.1.3.3 Update Setiap Neuron pada Layer.....	16

DAFTAR TABEL

Tabel 2.2.1.1	Tabel Pengaruh Width.....	17
Tabel 2.2.1.2	Tabel Perbandingan Visualisasi Perbandingan Pengaruh Width.....	23
Tabel 2.2.1.2	Tabel Pengaruh Width.....	24
Tabel 2.2.1.2	Tabel Perbandingan Visualisasi Perbandingan Pengaruh Depth.....	30
Tabel 2.2.2.1	Tabel Fungsi Aktivasi.....	30
Tabel 2.2.2.2	Tabel Perbandingan Visualisasi Perbandingan Fungsi Aktivasi.....	36
Tabel 2.2.3.1	Tabel Learning Rate.....	36
Tabel 2.2.3.2	Tabel Perbandingan Visualisasi Perbandingan Learning Rate.....	45
Tabel 2.2.4.1	Tabel Inisialisasi Bobot.....	45
Tabel 2.2.4.2	Tabel Perbandingan Visualisasi Perbandingan Inisialisasi Bobot...	52
Tabel 2.2.5.1	Tabel Regularisasi.....	53
Tabel 2.2.5.2	Tabel Perbandingan Visualisasi Regularisasi.....	59
Tabel 2.2.6.1	Tabel Perbandingan Sklearn.....	59
Tabel 2.2.6.2	Tabel Perbandingan Visualisasi dengan Sklearn.....	62
Tabel 4.1	Tabel Pembagian Tugas Kelompok.....	63

1. Deskripsi Persoalan

Tugas Besar 1 dalam mata kuliah IF3270 Pembelajaran Mesin bertujuan untuk melakukan segmentasi pelanggan berdasarkan karakteristik mereka menggunakan teknik klasifikasi. Dataset yang diberikan berisi informasi demografis dan perilaku pelanggan dengan berbagai fitur, seperti usia, pekerjaan, pengalaman kerja, tingkat pengeluaran, dan ukuran keluarga. Tugas ini mencakup eksplorasi data awal (EDA) untuk memahami pola yang terdapat dalam dataset serta pemodelan klasifikasi untuk mengelompokkan pelanggan ke dalam segmen tertentu. Model pembelajaran mesin yang digunakan diharapkan dapat memberikan wawasan yang bermanfaat bagi bisnis dalam menyesuaikan strategi pemasaran dan layanan terhadap kelompok pelanggan yang berbeda.

2. Pembahasan

Bagian ini berisi tentang penjelasan implementasi yang mendeskripsikan atribut serta method pada suatu kelas. Selain itu, terdapat penjelasan tentang forward propagation dan backward propagation, serta weight update. Untuk diperhatikan adalah bahwa kelas yang dipakai implementasi utama adalah FFNN.py. Kelas FFNN pada FFNNAutoDiff.py diimplementasikan namun tidak digunakan karena waktu training terlalu lambat.

2.1 Penjelasan implementasi

2.1.1 Deskripsi kelas beserta deskripsi atribut dan methodnya

- **FFNN.py**

No	Kelas	Atribut/Method	Deskripsi
1.	Neuron	<code>__call__(self, x)</code>	Memanggil neuron dengan input x
2.		<code>parameters(self)</code>	Mengembalikan parameter neuron
3.		<code>parameters_gradient(self)</code>	Method ini mengembalikan daftar nilai gradien dari parameter bobot (weight) dan bias (bias) dari neuron. Gradien ini menunjukkan seberapa besar perubahan bobot dan bias yang diperlukan untuk meminimalkan loss selama

			proses pelatihan.
4.		backward(self, gradient_output)	Method ini menghitung gradien dari bobot dan bias neuron berdasarkan error yang dihitung dalam backpropagation.
5.		update(self, learning_rate, batch_size)	Method ini memperbarui bobot dan bias neuron berdasarkan gradien yang telah dihitung dalam metode backward().
6.	Layer	__call__(self, x)	Memanggil layer dengan input x
7.		backward(self, gradient_output)	Method ini melakukan backpropagation pada seluruh neuron di dalam layer dan menghitung gradien input yang akan diteruskan ke layer sebelumnya.
8.		update(self, learning_rate, batch_size)	Method ini memperbarui bobot dan bias dari setiap neuron dalam layer menggunakan Gradient Descent.
9.		parameters_matrix(self)	Method ini mengembalikan matriks parameter (bobot & bias) dari semua neuron dalam layer.
10.		parameters_gradient_matrix(self)	Method ini mengembalikan matriks gradien dari parameter (bobot & bias) untuk semua neuron dalam layer.
11.	FFNN	weight_initializer(self, weight_initializer="zero", seed=0, lower_bound=-1, upper_bound=1, mean=0, variance=0.1)	Menginisialisasi bobot neuron dalam setiap layer dengan metode tertentu (zero, uniform, normal).
12.		reset_gradients(self)	Mengatur ulang gradien

			bobot dan bias di setiap neuron sebelum pembaruan bobot.
13.		encode_labels(self,y)	Mengonversi label kategori ke indeks numerik untuk keperluan training.
14.		decode_labels(self, y_pred)	Mengonversi prediksi numerik kembali ke label kategori.
15.		__call__(self,y)	Melakukan forward propagation melalui seluruh jaringan.
16.		compute_loss(self, y_true, y_pred)	Menghitung nilai loss berdasarkan fungsi loss yang dipilih (mse, binary_cross_entropy, categorical_cross_entropy).
17.		compute_loss_gradient(self, y_true, y_pred)	Menghitung gradien loss terhadap output model.
18.		fit(self, X, y, X_val=None, y_val=None)	Melatih model menggunakan algoritma backpropagation dan gradient descent dengan opsi validasi.
19.		predict_proba(self, X)	Menghasilkan probabilitas output untuk input yang diberikan.
20.		predict(self, X)	Menghasilkan prediksi kelas dari input berdasarkan probabilitas yang dihitung.
21.		parameters_matrix(self)	Mengembalikan bobot dan bias setiap neuron dalam bentuk matriks.
22.		parameters_gradient_matrix(self)	Mengembalikan gradien bobot dan bias setiap neuron dalam bentuk matriks.
23.		visualize_graph(self)	Membuat visualisasi arsitektur jaringan saraf

			menggunakan grafik.
		visualize_weight_distributions(self, layers)	Membuat histogram distribusi bobot dalam layer yang dipilih.
		visualize_weight_gradient_distributions(self, layers)	Membuat histogram distribusi gradien bobot dalam layer yang dipilih.
		visualize_loss(self)	Memvisualisasikan grafik loss selama training dan validasi.
		save(self, filename='model.pkl')	Menyimpan model yang telah dilatih ke dalam file .pkl.
		load(cls, filename)	Memuat model yang telah disimpan dari file .pkl.

- **AutoDiff.py**

No	Kelas	Atribut/Method	Deskripsi
1.	Value	updateGradients(self)	Menghitung gradien dengan melakukan propagasi mundur melalui computational graph menggunakan topological sorting.
2.		__add__(self, other)	Mengimplementasikan operasi penjumlahan antara dua objek Value , termasuk propagasi gradien ke nilai sebelumnya.
3.		_updatePreviousGradients()	Fungsi internal yang diperbarui untuk menghitung dan memperbarui gradien nilai sebelumnya dalam computational graph.
4.		__mul__(self, other)	Mengimplementasikan operasi perkalian dua objek Value dengan propagasi

			mundur menggunakan aturan rantai (chain rule).
5.		isinstance(other, Value)	Memeriksa apakah other adalah instance dari kelas Value , digunakan untuk memastikan kompatibilitas operasi aritmatika.
6.		__pow__(self, other)	Mengimplementasikan operasi perpangkatan dengan menghitung gradien berdasarkan diferensiasi eksponensial.
17.		__neg__(self)	Mengimplementasikan negasi suatu nilai dengan mengalikannya dengan -1 .
8.		exp(self)	Menghitung nilai eksponensial e^x dari objek Value , dengan gradien dihitung sebagai e^x .
9.		log(self)	Menghitung logaritma natural ($\ln(x)$) dari objek Value , dengan gradien dihitung sebagai $1/x$.
10.		relu(self)	Mengimplementasikan fungsi aktivasi ReLU (Rectified Linear Unit), yang mempertahankan nilai jika positif dan nol jika negatif.
11.		__radd__(self, other)	Mengimplementasikan operasi penjumlahan ketika Value berada di sebelah kanan operator + (misalnya 3 + Value).
12.		__sub__(self, other)	Mengimplementasikan operasi pengurangan dengan menggunakan penjumlahan dan negasi (self + (-other)).
13.		__rsub__(self, other)	Mengimplementasikan operasi pengurangan ketika

			Value berada di sebelah kanan operator - (misalnya 3 - Value).
14.		<code>__rmul__(self, other)</code>	Mengimplementasikan operasi perkalian ketika Value berada di sebelah kanan operator * (misalnya 3 * Value).
15.		<code>__truediv__(self, other)</code>	Mengimplementasikan operasi pembagian (self / other) menggunakan aturan self * other**-1 .
16.		<code>__rtruediv__(self, other)</code>	Mengimplementasikan operasi pembagian ketika Value berada di sebelah kanan operator / (misalnya 3 / Value).
17.		<code>__repr__(self)</code>	Mengembalikan representasi string dari objek Value , menampilkan nilai dan gradiennya.

- **FFNNAutoDiff.py**

No	Kelas	Atribut/Method	Deskripsi
1.	Neuron	<code>__call__(self, x)</code>	Melakukan perhitungan output neuron berdasarkan input x menggunakan bobot dan bias, lalu menerapkan fungsi aktivasi (linear, relu, sigmoid, tanh, softmax).
2.		<code>parameters(self)</code>	Mengembalikan daftar parameter neuron, yaitu bias dan bobot.
3.		<code>reset_gradient(self)</code>	Mengatur ulang gradien bobot dan bias menjadi nol sebelum pembaruan bobot.
4.	Layer	<code>__call__(self, x)</code>	Melakukan perhitungan forward pass dengan memberikan input x ke

			setiap neuron dalam layer. Jika hanya ada satu neuron, mengembalikan output tunggal, jika lebih dari satu, mengembalikan daftar output.
5.		parameters_matrix(self)	Mengembalikan daftar parameter (bobot dan bias) dari semua neuron dalam layer dalam bentuk matriks.
6.		parameters(self)	Mengembalikan daftar parameter (bobot dan bias) dari semua neuron dalam layer dalam bentuk satu dimensi.
7.		reset_gradient(self)	Mengatur ulang gradien bobot dan bias semua neuron dalam layer menjadi nol sebelum pembaruan bobot.
8.	FFNN	encode_labels(self, y)	Mengonversi label kelas menjadi bentuk numerik untuk keperluan pembelajaran.
9.		decode_labels(self, encoded_labels)	Mengembalikan label numerik ke bentuk aslinya setelah prediksi.
10.		weight_initializer(self, weight_initializer="zero", seed=0, lower_bound=-1, upper_bound=1, mean=0, variance=0.1)	Menginisialisasi bobot dan bias untuk setiap neuron dalam layer dengan berbagai metode (zero, uniform, normal).
11.		__call__(self, y)	Melakukan forward propagation dengan memberikan input ke seluruh layer dan menghasilkan output prediksi.
12.		parameters_matrix(self)	Mengembalikan bobot dan bias dalam bentuk matriks untuk setiap layer.

13.		parameters(self)	Mengembalikan semua bobot dan bias dalam bentuk daftar satu dimensi.
14.		reset_gradient(self)	Mengatur ulang gradien bobot dan bias sebelum melakukan pembaruan parameter.
15.		fit(self, X, y)	Melatih model menggunakan data latih dengan metode backpropagation dan gradient descent.
16.		predict(self, X)	Melakukan prediksi terhadap data baru dan mengembalikan label hasil klasifikasi.
17.		predict_proba(self, X)	Menghasilkan probabilitas untuk setiap kelas dalam prediksi.
18.		save(self, filename='model.pkl')	Menyimpan model dalam file .pkl menggunakan pickle.
19.		load(cls, filename)	Memuat model yang telah disimpan dalam file .pkl.
20.		visualize_graph(self)	Membuat visualisasi arsitektur jaringan menggunakan Graphviz.
21.		visualize_weight_distribution(self, layer)	Memvisualisasikan distribusi bobot di layer tertentu menggunakan histogram.
22.		visualize_weight_gradient_distribution(self, layer)	Memvisualisasikan distribusi gradien bobot di layer tertentu menggunakan histogram.

- **Utils.py**

No	Atribut/Method	Deskripsi
----	----------------	-----------

1.	<code>topological_sort(root)</code>	Menentukan urutan perhitungan gradien dalam proses backpropagation pada jaringan saraf berbasis komputasi graf.
----	-------------------------------------	---

2.1.2 Penjelasan forward propagation

Forward propagation adalah proses penghitungan dan pengiriman informasi melalui jaringan saraf tiruan dari lapisan masukan (input layer) hingga lapisan keluaran (output layer). Proses ini melibatkan serangkaian operasi matematika yang dilakukan pada setiap lapisan jaringan. Tujuan utama dari *forward propagation* adalah untuk menghasilkan prediksi berdasarkan input yang diberikan. Prediksi ini kemudian dibandingkan dengan nilai target yang sebenarnya untuk menghitung kesalahan (error).

```
outputs = np.array([self(x) for x in X_batch])
loss = self.compute_loss(y_batch, outputs)
total_loss += loss
```

Gambar 2.1.2.1 Total Loss

Outputs akan menyimpan hasil forward propagation untuk setiap input *x* dalam *X_batch* dengan melakukan pemanggilan method `__call__` pada class FFNN. Hasil *outputs* akan digunakan untuk menghitung *loss* untuk batch saat ini, kemudian diakumulasikan pada *total_loss*.

```
def __call__(self, x):
    for layer in self.layers:
        x = layer(x)
    return x
```

Gambar 2.1.2.2 Method Call Kelas FFNN

Method ini akan melakukan forward propagation dengan melakukan pemanggilan method `__call__` pada class Layer untuk setiap layer yang ada pada FFNN dengan sebuah input *x*.

```
class Layer():
    def __init__(self, n_input, n_output, activation_function="linear"):
        self.neurons = [Neuron(n_input, activation_function) for _ in
                           range(n_output)]

    def __call__(self, x):
        return np.array([n(x) for n in self.neurons])
```

Gambar 2.1.2.2 Method Call Kelas Layer

Method `__call__` pada class Layer akan melakukan pemanggilan method `__call__` pada class Neuron untuk setiap Neuron yang ada pada Layer tersebut dengan input `x`.

```
def __call__(self, x):
    self.input = np.array(x)
    z = np.dot(self.weight, self.input) + self.bias

    if self.activation_function == "linear":
        self.output = z
    elif self.activation_function == "relu":
        self.output = np.maximum(0, z)
    elif self.activation_function == "sigmoid":
        self.output = 1 / (1 + np.exp(-z))
    elif self.activation_function == "tanh":
        self.output = np.tanh(z)
    elif self.activation_function == "softmax":
        exp_z = np.exp(z - np.max(z))
        self.output = exp_z / exp_z.sum()
    return self.output
```

Gambar 2.1.2.3 Method Call Kelas Neuron

Method `__call__` pada class Neuron akan melakukan perkalian matriks antara input dan bobot pada neuron tersebut, kemudian ditambahkan dengan bobot bias. Hasil perkalian tersebut akan diaktivasi dengan memanggil fungsi aktivasi sesuai dengan fungsi aktivasi yang ditetapkan untuk neuron tersebut.

2.1.3 Penjelasan Backward Propagation dan Weight Update

Backward propagation adalah algoritma yang digunakan untuk menghitung gradien (turunan) dari fungsi kesalahan (loss function) terhadap bobot jaringan

neuron tiruan. Gradien ini kemudian digunakan untuk memperbarui bobot agar jaringan dapat membuat prediksi yang lebih akurat.

Weight update adalah proses penyesuaian bobot jaringan neuron tiruan berdasarkan gradien yang dihitung oleh *backward propagation*. Tujuannya adalah untuk meminimalkan kesalahan prediksi jaringan.

```
for j, x in enumerate(X_batch):
    gradient = gradients[j]
    for layer in reversed(self.layers):
        gradient = layer.backward(gradient)
    for layer in self.layers:
        layer.update(self.learning_rate, self.batch_size)
```

Gambar 2.1.3.1 Weight Update

Pada bagian ini yang terdapat pada method *fit* pada class FFNN, *gradients* akan menyimpan hasil gradient dari loss function untuk setiap sample. Setelah itu, gradient dari masing-masing sample akan di-*backpropagation* mulai dari layer yang terakhir dengan memanggil method *backward* pada layer. Hasil gradient pada layer tersebut akan digunakan untuk perhitungan pada layer sebelumnya.

```
def backward(self, gradient_output):
    gradient_input = np.zeros(len(self.neurons[0].weight), dtype=float)
    for i, neuron in enumerate(self.neurons):
        neuron.backward(gradient_output[i])
        gradient_input += neuron.weight * neuron.gradient_delta
    return gradient_input
```

Gambar 2.1.3.2 Method Backward Kelas Layer

Method *backward* pada class layer akan melakukan pemanggilan method *backward* untuk setiap neuron yang ada pada layer tersebut. Selain itu, pada bagian ini juga akan menyimpan penjumlahan dari hasil perkalian dari bobot-bobot pada suatu neuron dengan delta gradient neuron. Hasil akumulasi tersebut yang akan digunakan untuk perhitungan gradient pada layer sebelumnya.

```
def backward(self, gradient_output):
    if self.activation_function == "relu":
        self.gradient_delta = gradient_output * (self.output > 0)
    elif self.activation_function == "sigmoid":
        self.gradient_delta = gradient_output * (self.output * (1 - self.output))
    elif self.activation_function == "tanh":
        self.gradient_delta = gradient_output * (1 - self.output ** 2)
    elif self.activation_function == "softmax":
        self.gradient_delta = gradient_output * self.output * (np.eye(len(self.output)) - self.output.T)
    else:
        self.gradient_delta = gradient_output

    self.gradient_weight += self.gradient_delta * self.input
    self.gradient_bias += self.gradient_delta
```

Gambar 2.1.3.2 Method Backward Kelas Neuron

Method *backward* pada class Neuron akan menghitung gradient delta untuk neuron dengan mengalikan turunan fungsi aktivasi dengan gradien output. Gradient delta ini akan digunakan untuk chain rule saat menghitung gradient delta neuron sebelumnya. Pada bagian ini juga akan dihitung gradient weight dan bias yang akan digunakan untuk mengupdate bobot.

Setelah selesai melakukan backpropagation, barulah dilakukan update weight untuk setiap layer dengan memanggil method update pada class Layer.

```
def update(self, learning_rate, batch_size):
    for neuron in self.neurons:
        neuron.update(learning_rate, batch_size)
```

Gambar 2.1.3.3 Update Kelas Layer

Method ini akan memanggil method update untuk setiap neuron pada layer tersebut.

```
def update(self, learning_rate, batch_size):
    self.weight -= learning_rate * self.gradient_weight / batch_size
    self.bias -= learning_rate * self.gradient_bias / batch_size
```

Gambar 2.1.3.3 Update Setiap Neuron pada Layer

Method update pada class Neuron akan mengupdate semua weight and bias pada neuron tersebut.

2.2 Hasil pengujian

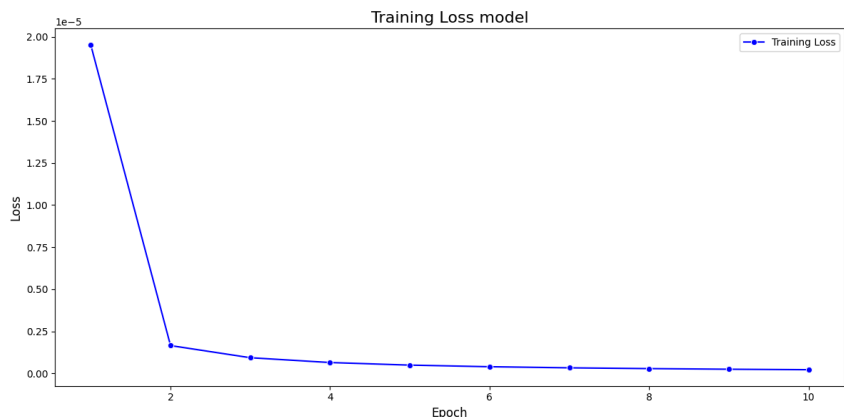
2.2.1 Pengaruh depth dan width

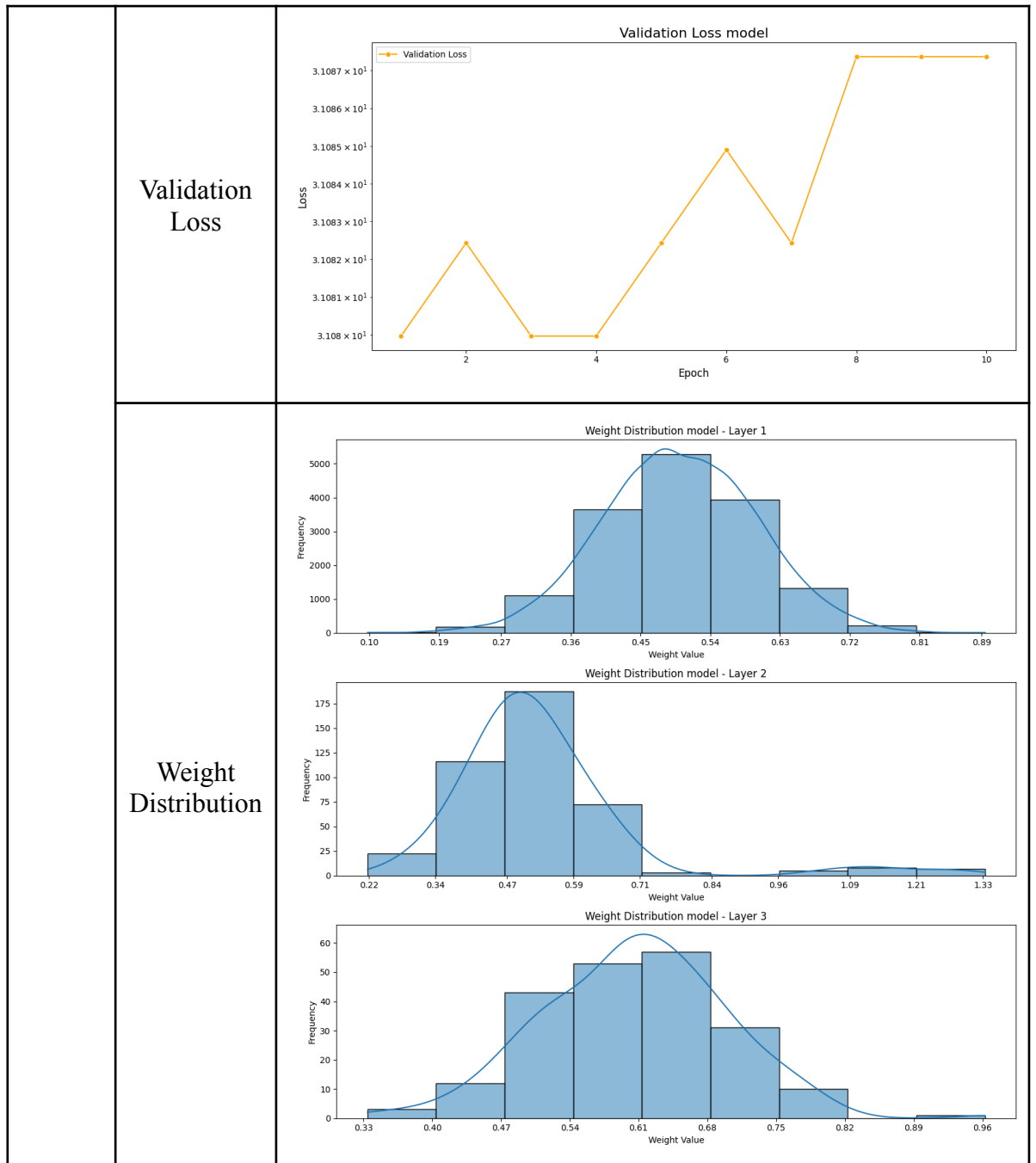
A. Pengaruh Width

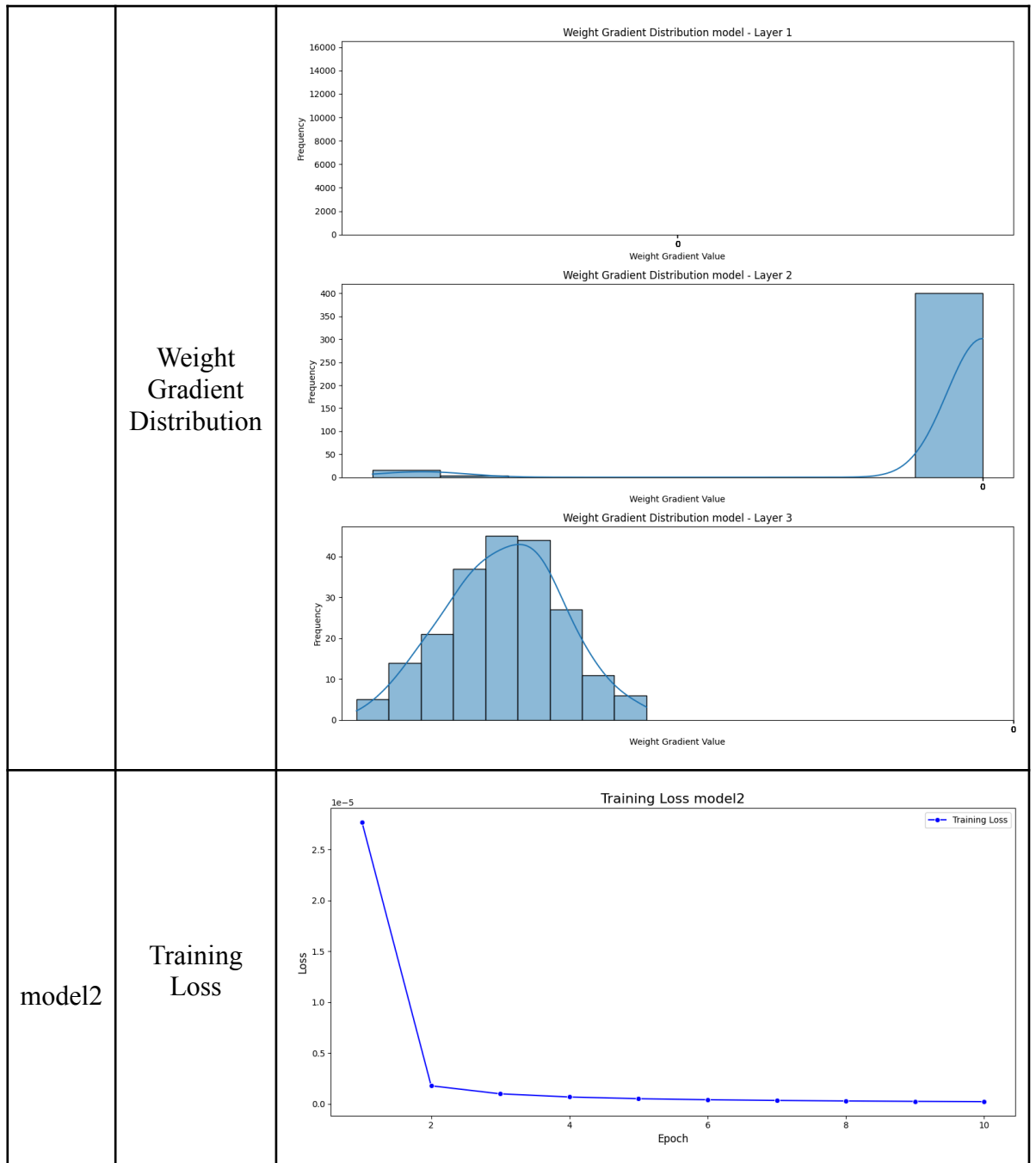
Dilakukan percobaan dengan 3 variasi width dengan 2 hidden layer, yaitu model1 dengan masing-masing hidden layer 20 neuron, model2 dengan masing-masing hidden layer 15 neuron, dan model3 dengan masing-masing hidden layer 10 neuron.

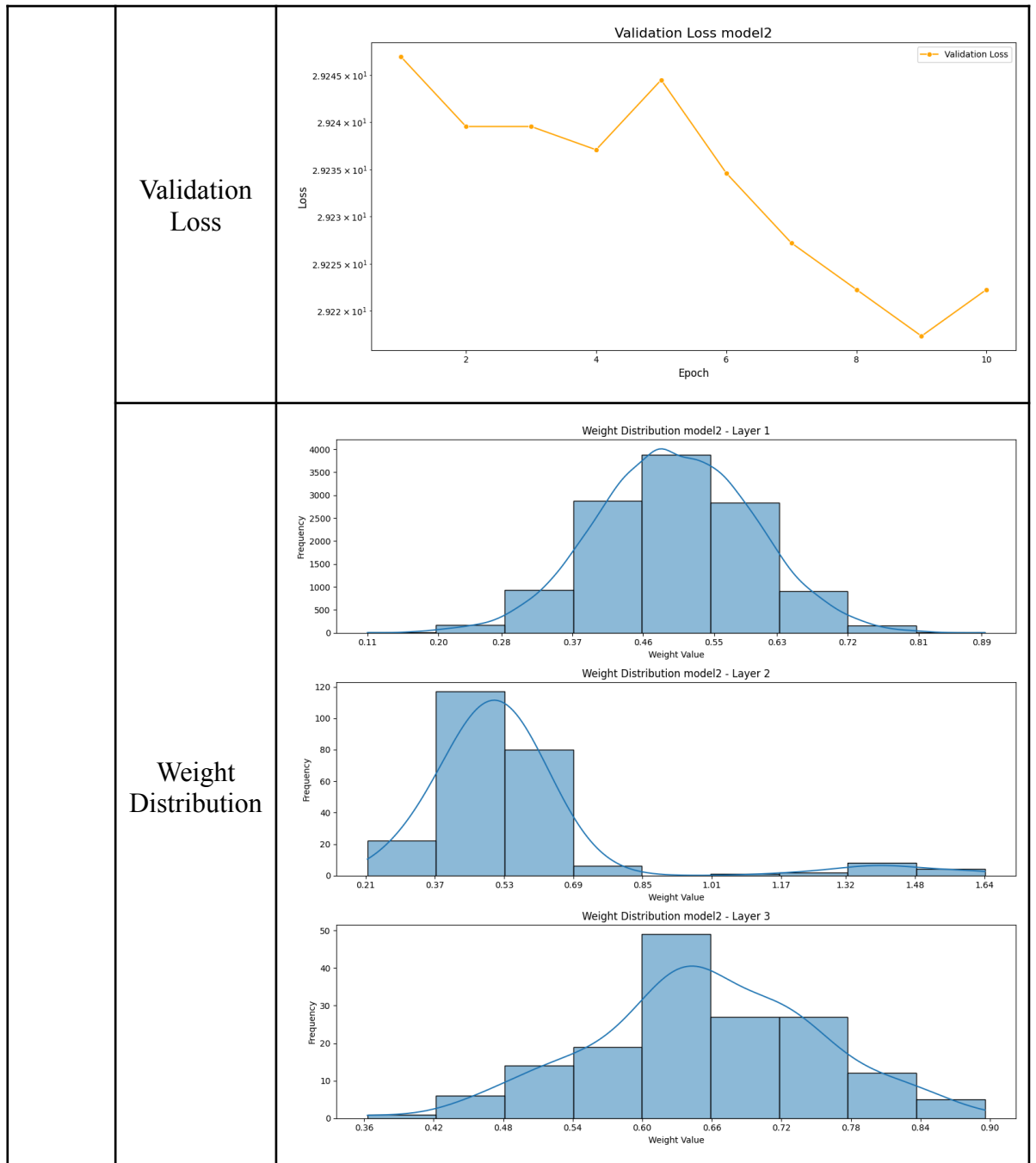
Model	Neuron per Hidden Layer	Score Prediksi
model1	20	0.0999
model2	15	0.1539
model3	10	0.1247

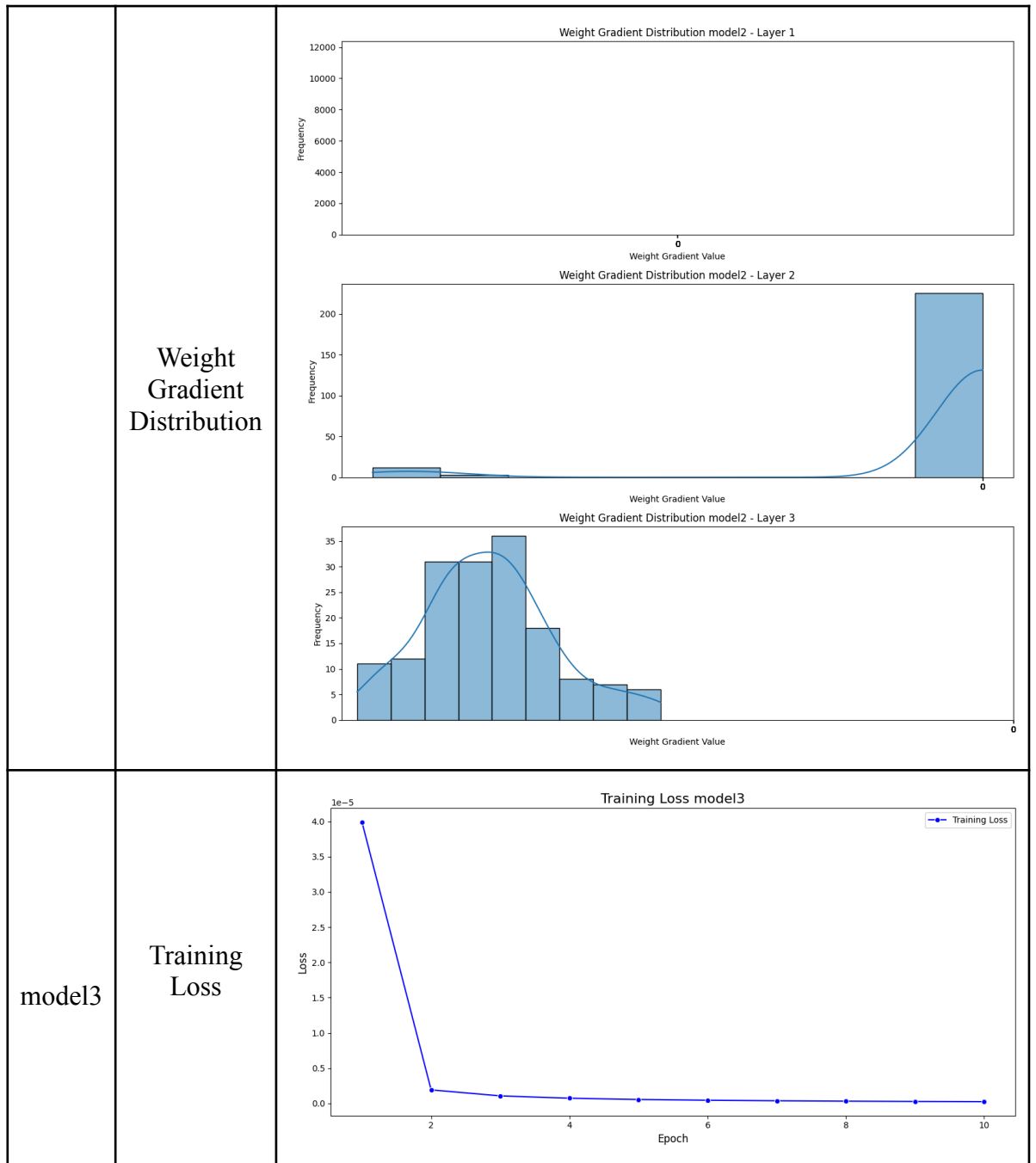
Tabel 2.2.1.1 Tabel Pengaruh Width

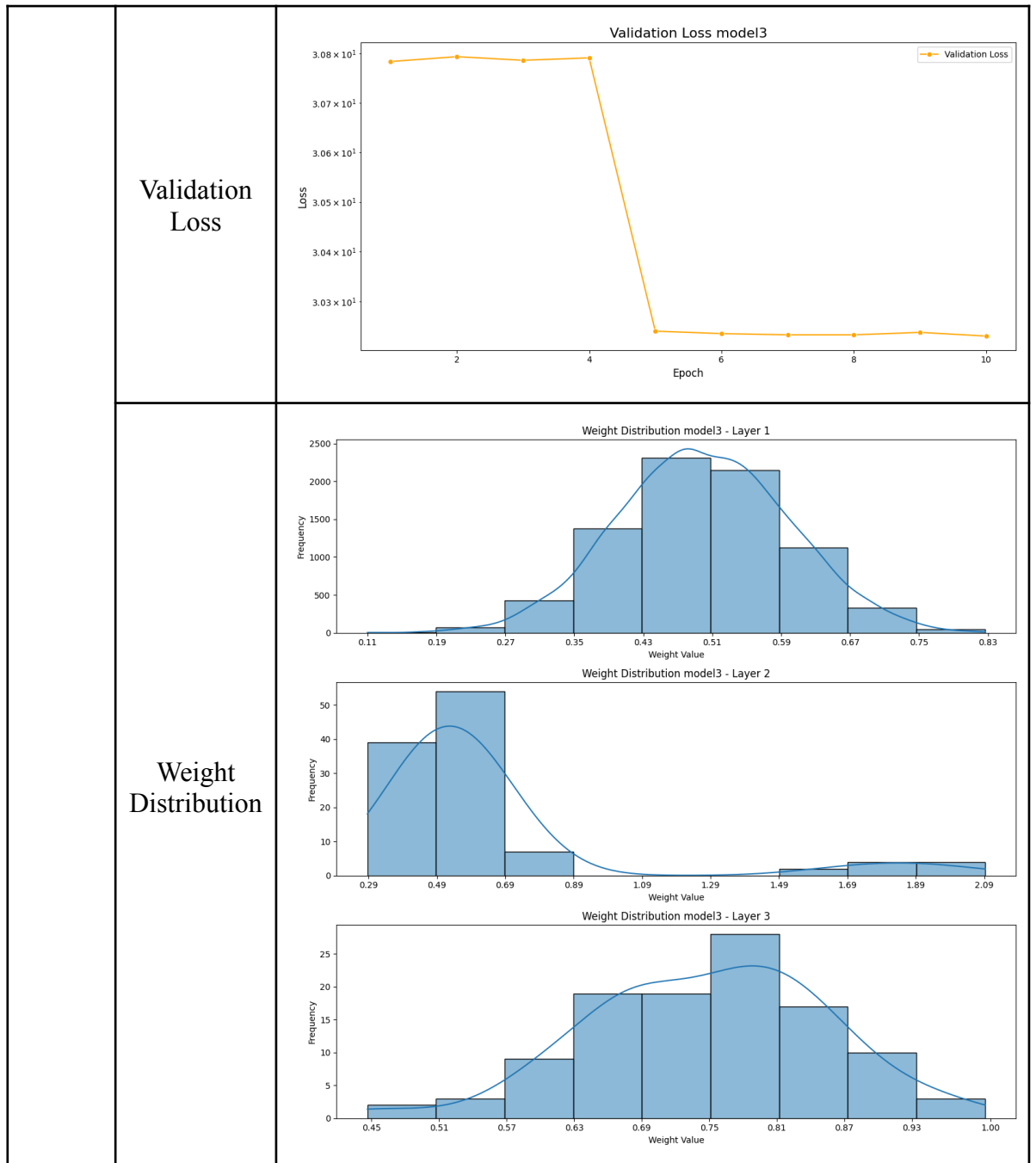
Model	Visualization																					
model1	Training Loss																					
		<table><caption>Training Loss Data (Estimated)</caption><tr><th>Epoch</th><th>Training Loss (x 10⁻⁵)</th></tr><tr><td>1</td><td>1.90</td></tr><tr><td>2</td><td>0.15</td></tr><tr><td>3</td><td>0.08</td></tr><tr><td>4</td><td>0.05</td></tr><tr><td>5</td><td>0.03</td></tr><tr><td>6</td><td>0.02</td></tr><tr><td>7</td><td>0.01</td></tr><tr><td>8</td><td>0.01</td></tr><tr><td>9</td><td>0.01</td></tr><tr><td>10</td><td>0.01</td></tr></table>	Epoch	Training Loss (x 10 ⁻⁵)	1	1.90	2	0.15	3	0.08	4	0.05	5	0.03	6	0.02	7	0.01	8	0.01	9	0.01
Epoch	Training Loss (x 10 ⁻⁵)																					
1	1.90																					
2	0.15																					
3	0.08																					
4	0.05																					
5	0.03																					
6	0.02																					
7	0.01																					
8	0.01																					
9	0.01																					
10	0.01																					

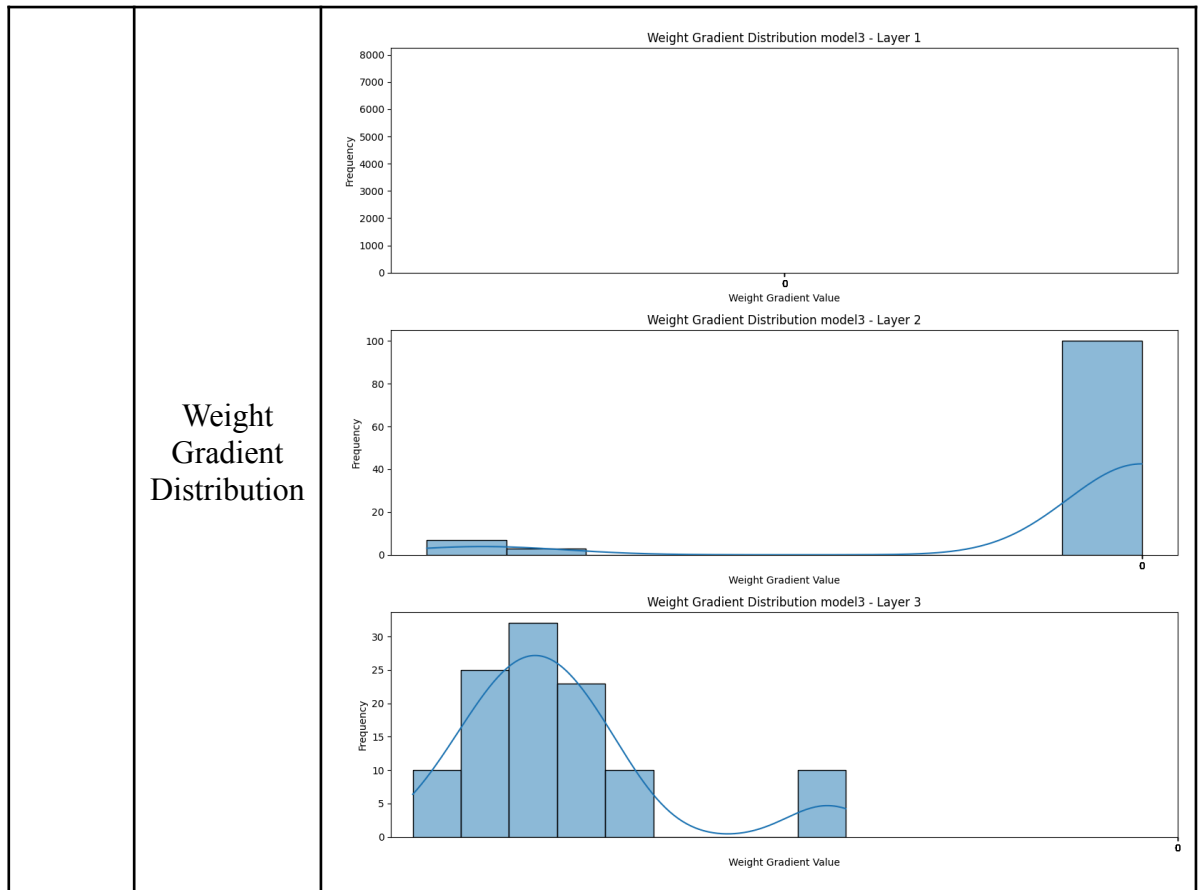












Tabel 2.2.1.2 Tabel Perbandingan Visualisasi Perbandingan Pengaruh Width

Grafik training loss untuk ketiga model mengalami penurunan seiring bertambahnya epoch. Grafik validation loss untuk model1 mengalami kenaikan dan penurunan dengan penurunan terdalam pada epoch 6, sebelum naik kembali. Sedangkan pada model2, grafik validation loss mengalami penurunan hingga epoch 8, sebelum naik kembali. Pada model3, grafik validation loss konsisten mengalami penurunan. Jika diurutkan berdasarkan validation loss pada akhir pelatihan model2 memiliki validation loss paling kecil, diikuti oleh model3 dan model 1.

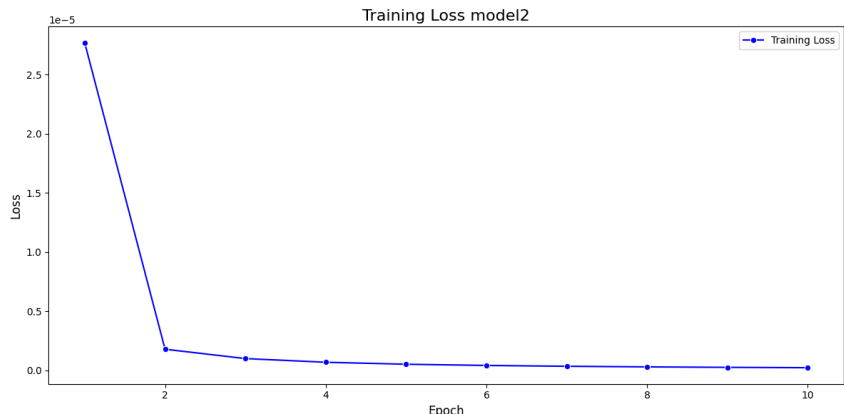
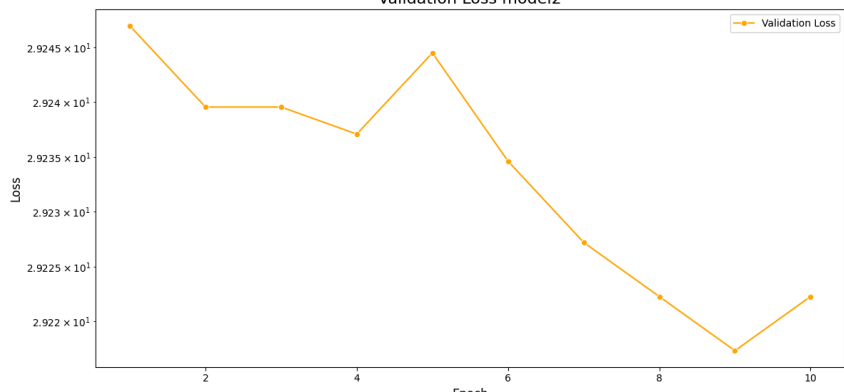
Dari hasil percobaan, terlihat bahwa penambahan neuron dapat meningkatkan score prediksi dan menghasilkan validation loss yang lebih kecil. Namun, jika neuron terlalu banyak akan menyebabkan overfitting. Model1 memiliki training loss yang paling kecil, namun memiliki score prediksi yang paling kecil dan validation loss yang paling besar. Model2 dengan 15 neuron memiliki score prediksi yang paling besar. Model2 juga memiliki training loss dan validation loss yang lebih kecil dibandingkan model3.

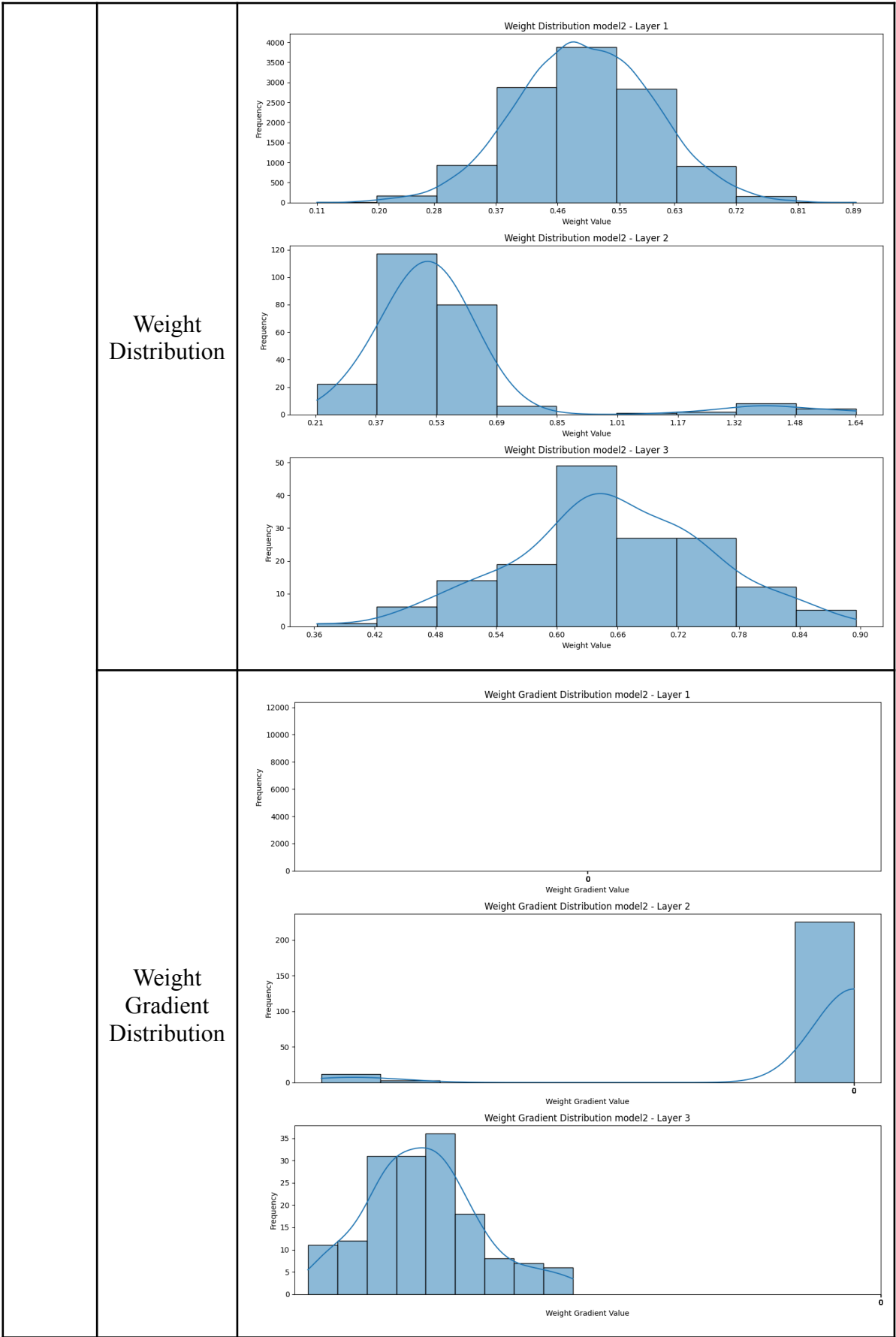
B. Pengaruh Depth

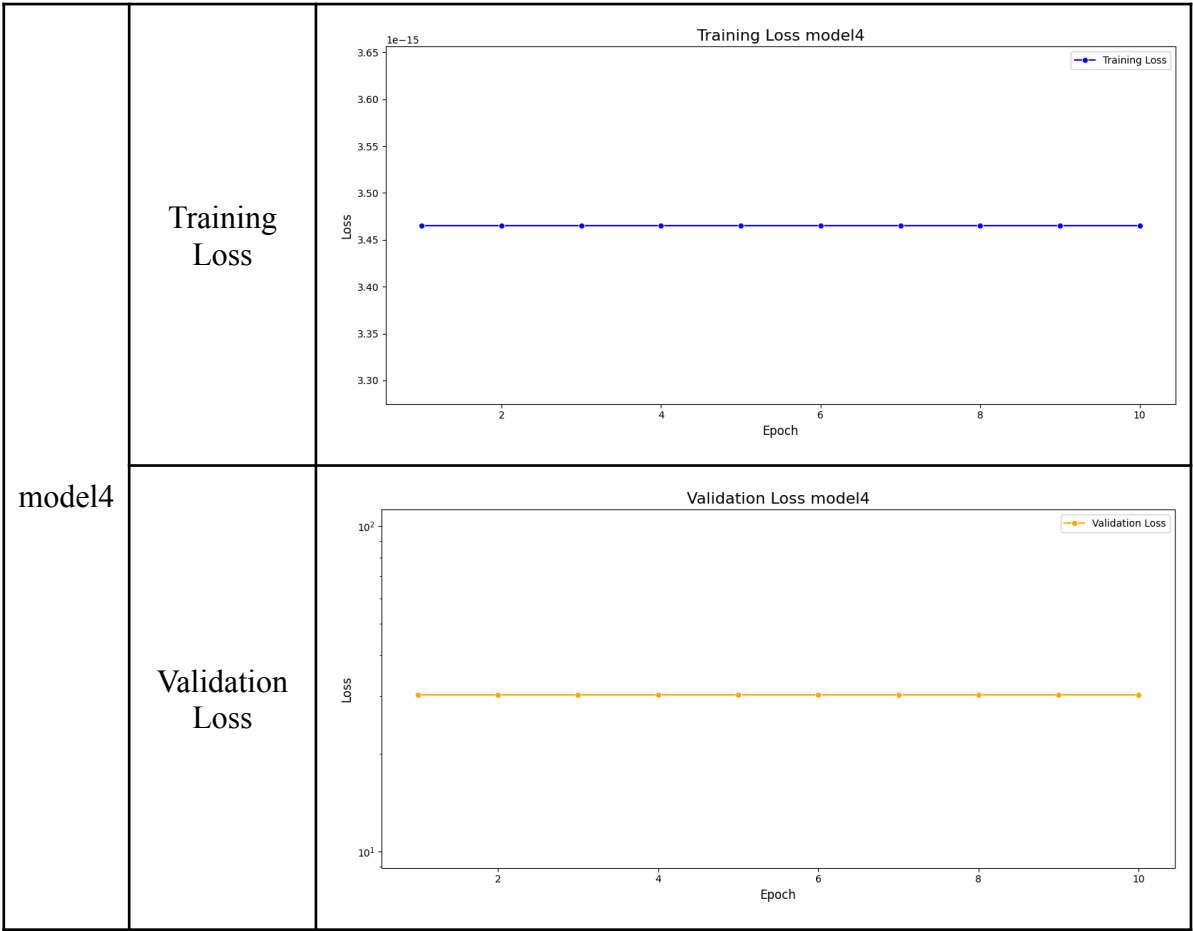
Dilakukan percobaan dengan 3 variasi depth dengan width hidden layer yang sama, yaitu 15 neuron. Model2 memiliki 2 hidden layer, model4 memiliki 3 hidden layer, dan model5 memiliki 1 hidden layer.

Model	Depth (hidden layer)	Score Prediksi
model2	2	0.1539
model4	3	0.1239
model5	1	0.1475

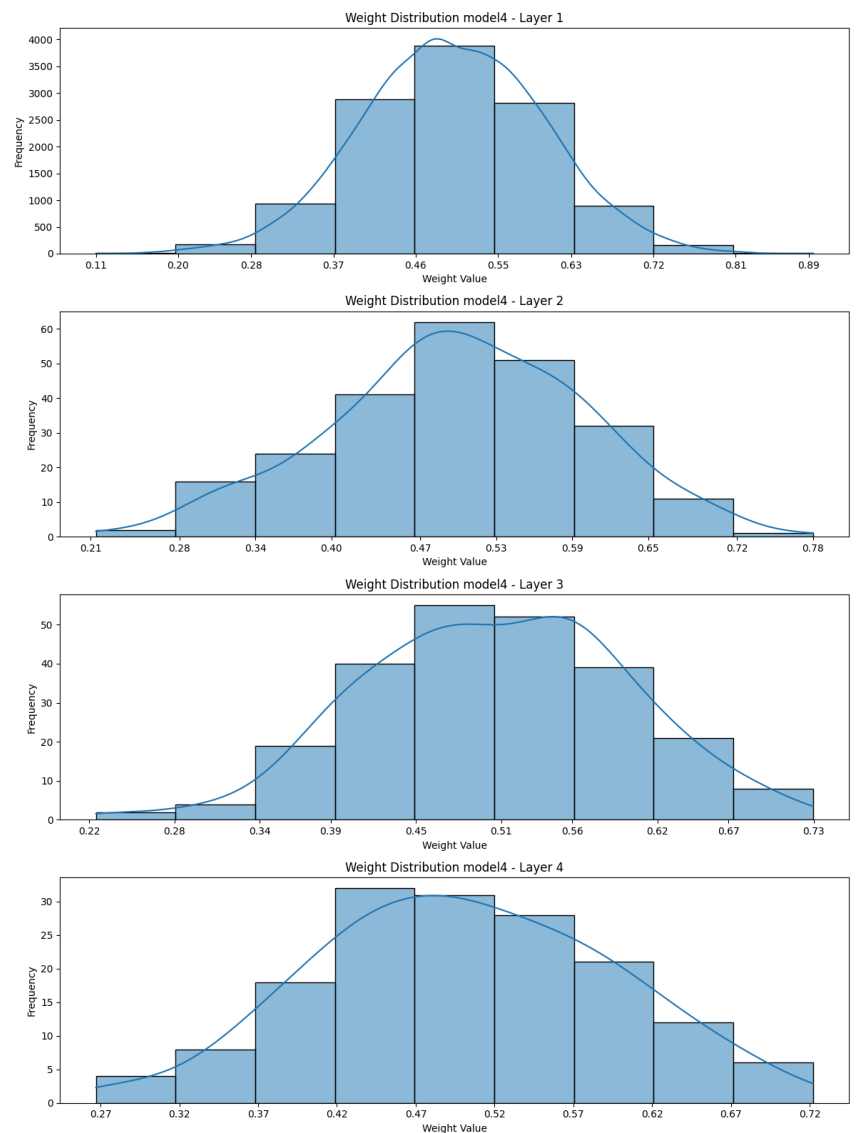
Tabel 2.2.1.2 Tabel Pengaruh Width

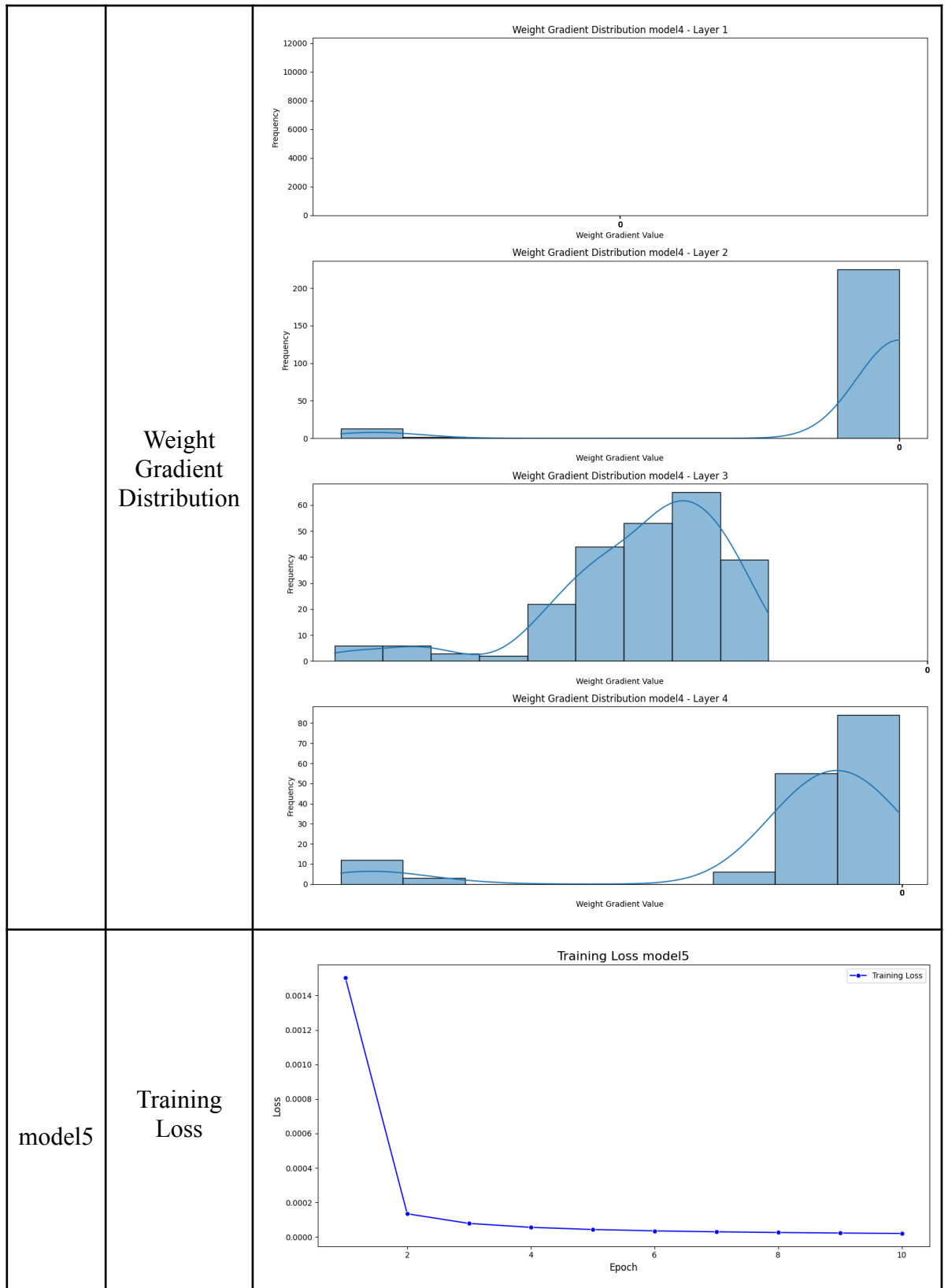
Model	Visualization																							
model2	Training Loss	 <table><caption>Training Loss model2 Data</caption><thead><tr><th>Epoch</th><th>Training Loss (x 10⁻⁵)</th></tr></thead><tbody><tr><td>1</td><td>2.8</td></tr><tr><td>2</td><td>0.2</td></tr><tr><td>3</td><td>0.1</td></tr><tr><td>4</td><td>0.08</td></tr><tr><td>5</td><td>0.06</td></tr><tr><td>6</td><td>0.05</td></tr><tr><td>7</td><td>0.04</td></tr><tr><td>8</td><td>0.03</td></tr><tr><td>9</td><td>0.02</td></tr><tr><td>10</td><td>0.01</td></tr></tbody></table>	Epoch	Training Loss (x 10 ⁻⁵)	1	2.8	2	0.2	3	0.1	4	0.08	5	0.06	6	0.05	7	0.04	8	0.03	9	0.02	10	0.01
	Epoch	Training Loss (x 10 ⁻⁵)																						
1	2.8																							
2	0.2																							
3	0.1																							
4	0.08																							
5	0.06																							
6	0.05																							
7	0.04																							
8	0.03																							
9	0.02																							
10	0.01																							
Validation Loss	 <table><caption>Validation Loss model2 Data</caption><thead><tr><th>Epoch</th><th>Validation Loss (x 10¹)</th></tr></thead><tbody><tr><td>1</td><td>2.9245</td></tr><tr><td>2</td><td>2.9240</td></tr><tr><td>3</td><td>2.9240</td></tr><tr><td>4</td><td>2.9235</td></tr><tr><td>5</td><td>2.9245</td></tr><tr><td>6</td><td>2.9230</td></tr><tr><td>7</td><td>2.9225</td></tr><tr><td>8</td><td>2.9220</td></tr><tr><td>9</td><td>2.9215</td></tr><tr><td>10</td><td>2.9220</td></tr></tbody></table>	Epoch	Validation Loss (x 10 ¹)	1	2.9245	2	2.9240	3	2.9240	4	2.9235	5	2.9245	6	2.9230	7	2.9225	8	2.9220	9	2.9215	10	2.9220	
Epoch	Validation Loss (x 10 ¹)																							
1	2.9245																							
2	2.9240																							
3	2.9240																							
4	2.9235																							
5	2.9245																							
6	2.9230																							
7	2.9225																							
8	2.9220																							
9	2.9215																							
10	2.9220																							

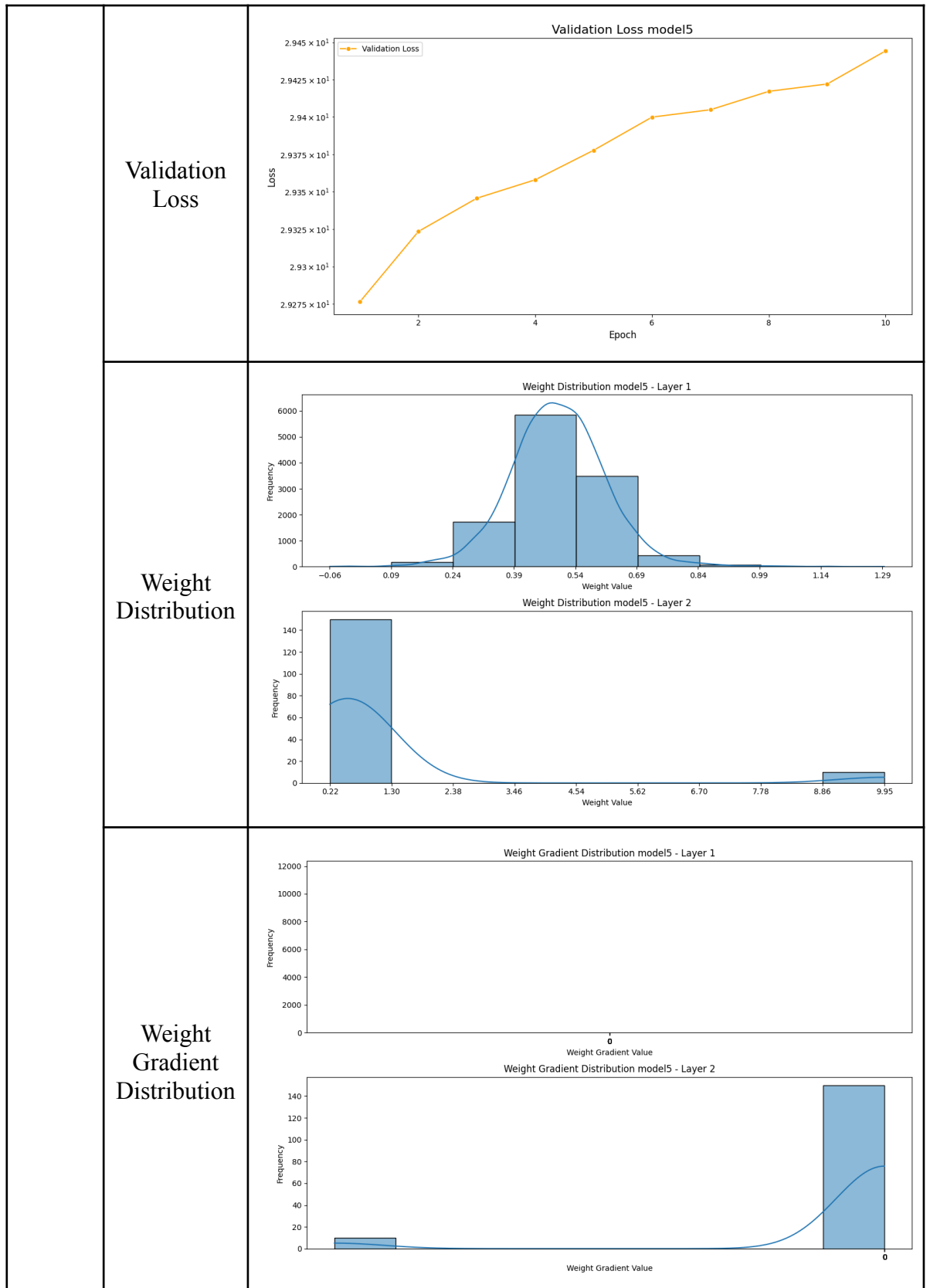




Weight Distribution







Tabel 2.2.1.2 Tabel Perbandingan Visualisasi Perbandingan Pengaruh Depth

Pada model2 dan model5 grafik training loss mengalami penurunan seiring bertambahnya epoch, sedangkan pada model4, grafik training loss dan validation loss cenderung stagnan. Pada model2, grafik validation loss mengalami penurunan hingga epoch 8, sebelum naik kembali. Sedangkan pada model5, grafik validation loss mempunyai trend menurun seiring bertambahnya epoch.

Dari hasil percobaan dapat terlihat bahwa penambahan depth akan meningkatkan score prediksi dan validation loss yang lebih kecil, namun jika depth terlalu banyak juga dapat menyebabkan overfitting. Pada model4 dengan 3 hidden layer mempunyai training loss yang paling kecil, namun validation loss lebih besar dari model2 dengan 2 hidden layer.

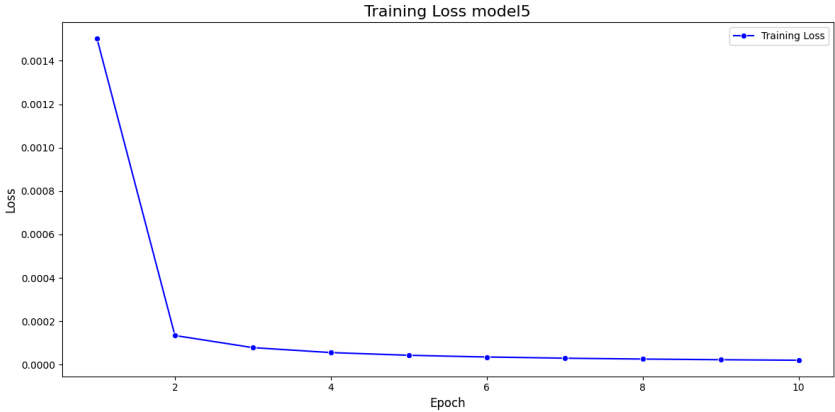
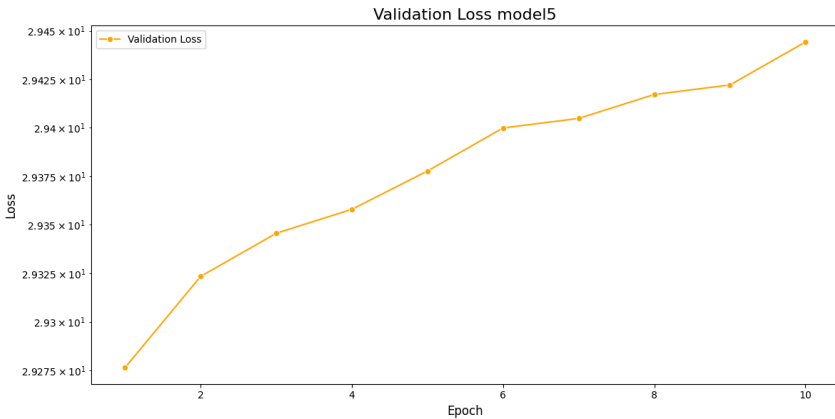
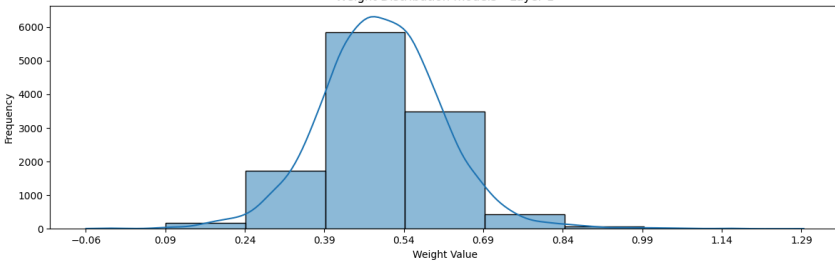
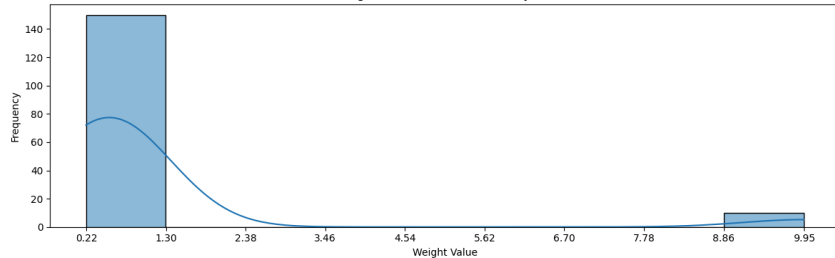
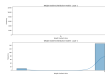
2.2.2 Pengaruh fungsi aktivasi

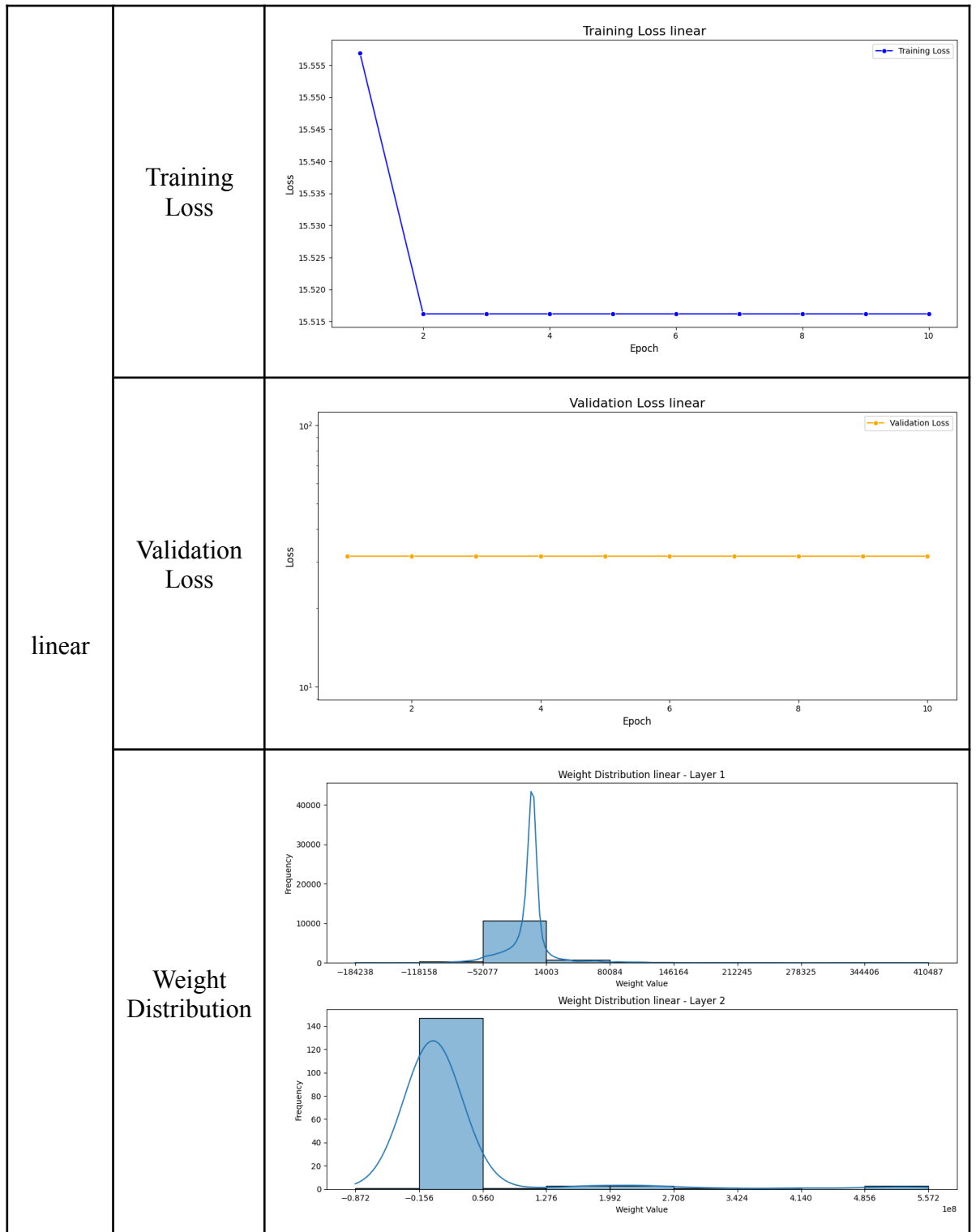
Dilakukan percobaan untuk setiap fungsi aktivasi, kecuali *softmax*. Percobaan akan dilakukan untuk semua fungsi aktivasi dengan 1 hidden layer dan fungsi aktivasi untuk output layer adalah sigmoid.

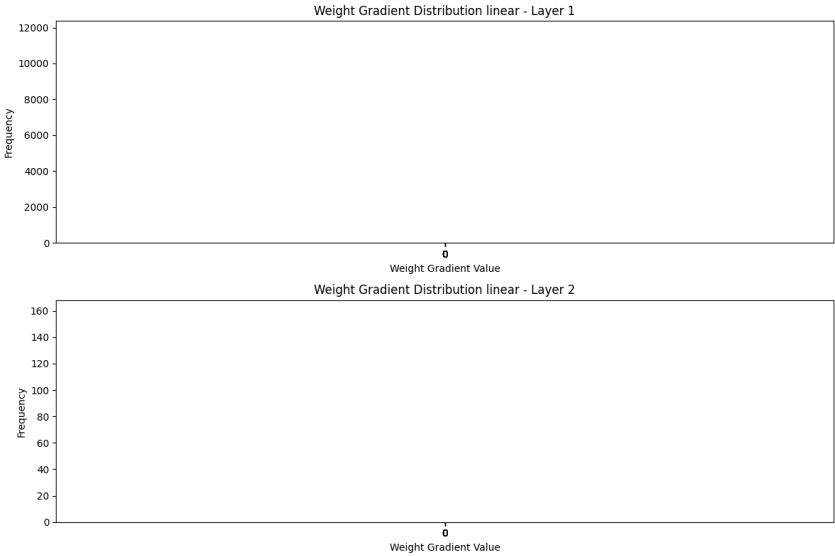
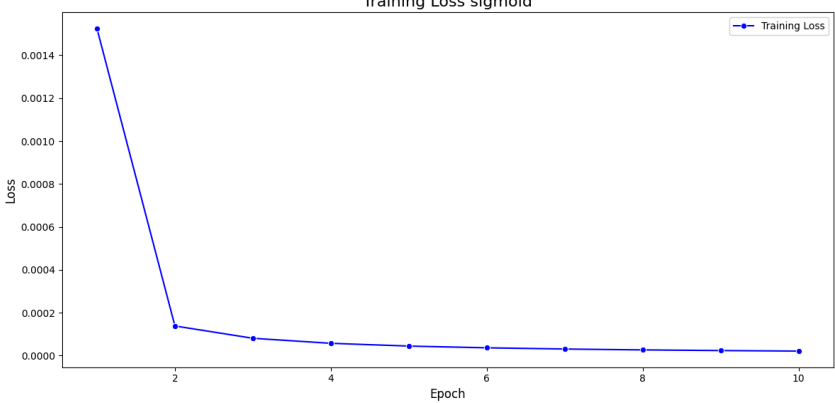
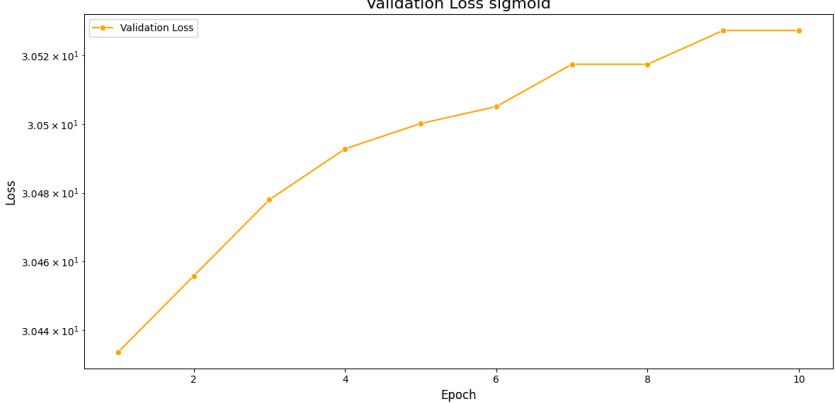
Model	Activation Function	Score Prediksi
model5	relu	0.1475
linear	linear	0.0874
sigmoid	sigmoid	0.1161
tanh	tanh	0.0896

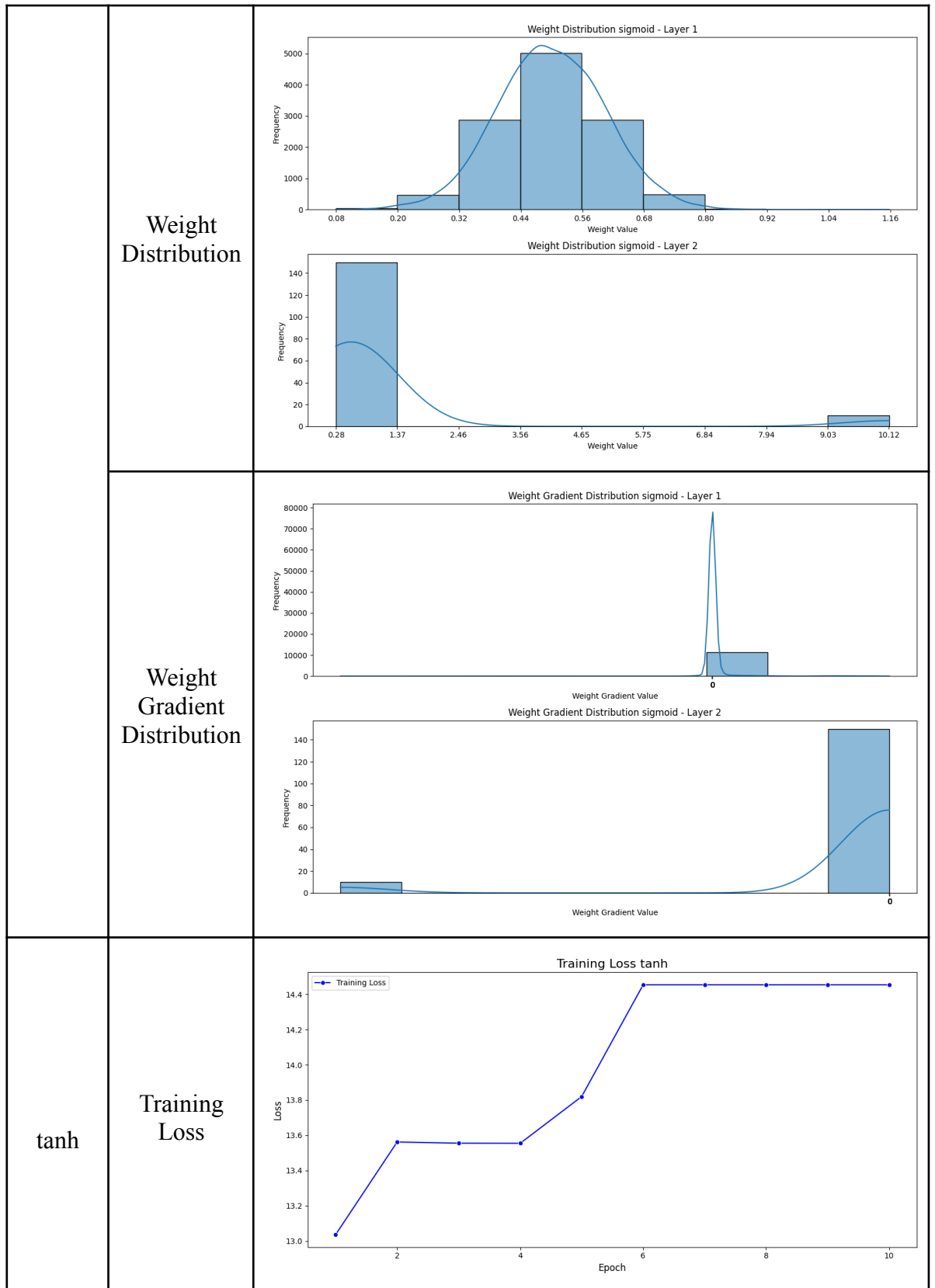
Tabel 2.2.2.1 Tabel Fungsi Aktivasi

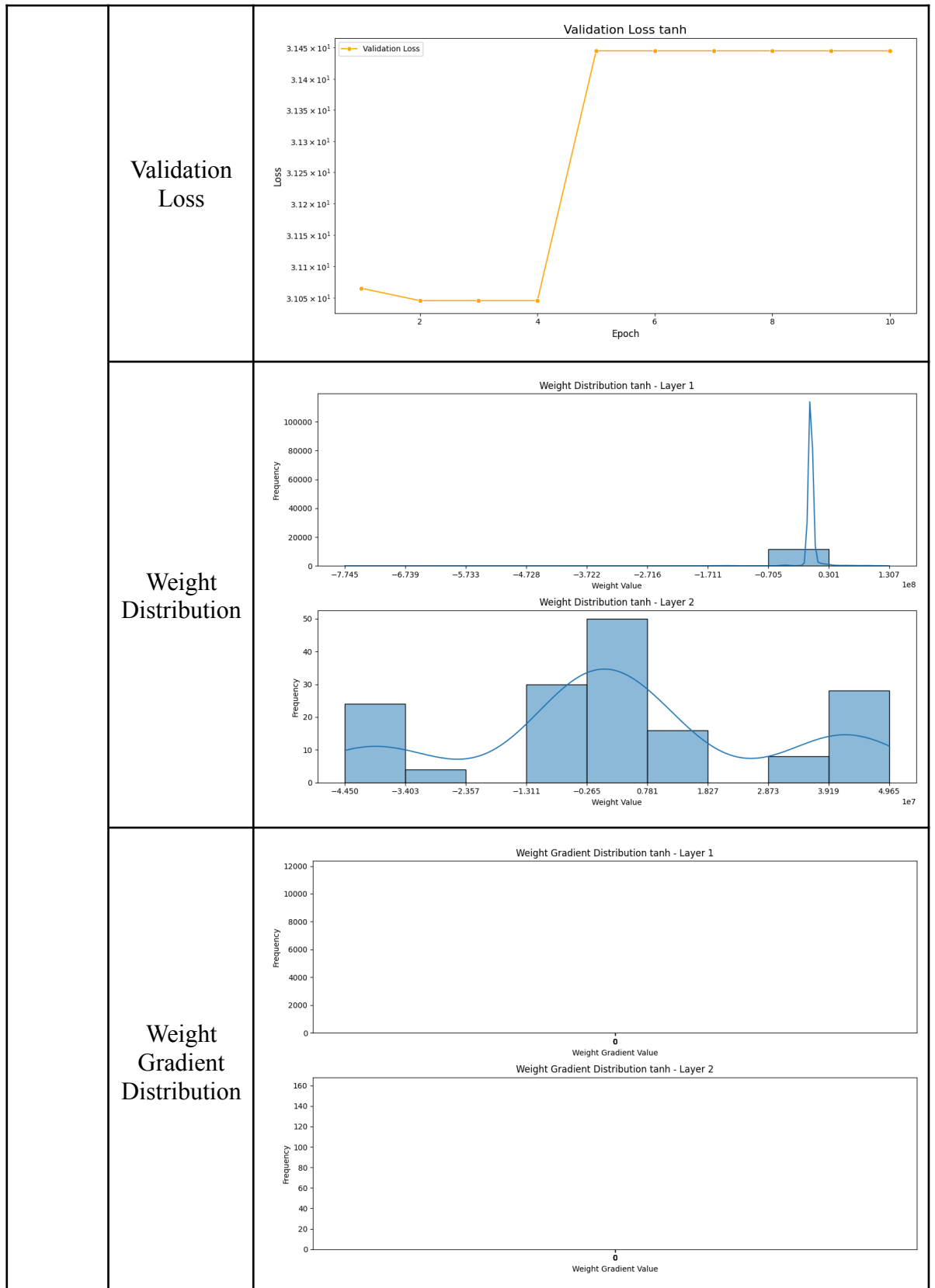
Model	Visualization
-------	---------------

model5	Training Loss	 Training Loss model5 Y-axis: Loss (0.0000 to 0.0014) X-axis: Epoch (1 to 10) Legend: Training Loss
	Validation Loss	 Validation Loss model5 Y-axis: Loss (2.9275 × 10¹ to 2.945 × 10¹) X-axis: Epoch (1 to 10) Legend: Validation Loss
	Weight Distribution	 Weight Distribution model5 - Layer 1 Y-axis: Frequency (0 to 6000) X-axis: Weight Value (-0.06 to 1.29)  Weight Distribution model5 - Layer 2 Y-axis: Frequency (0 to 140) X-axis: Weight Value (0.22 to 9.95)
	Weight Gradient Distribution	



	Weight Gradient Distribution	 Weight Gradient Distribution linear - Layer 1 Weight Gradient Distribution linear - Layer 2
sigmoid	Training Loss	 Training Loss sigmoid
	Validation Loss	 Validation Loss sigmoid





Tabel 2.2.2.2 Tabel Perbandingan Visualisasi Perbandingan Fungsi Aktivasi

Semua model memiliki grafik training loss yang cenderung menurun, kecuali model tanh yang memiliki grafik training loss meningkat. Model5 dengan fungsi aktivasi relu memiliki grafik validation loss yang menurun, model dengan fungsi aktivasi tanh dan linear memiliki grafik validation loss yang cenderung datar. Sedangkan model dengan fungsi aktivasi sigmoid memiliki grafik validation loss yang meningkat. Jika diurutkan dari training loss yang paling besar ke yang paling kecil adalah model linear, model tanh, model sigmoid, dan model5 (relu).

Distribusi bobot mencerminkan seberapa efektif pembelajaran yang terjadi dalam model. Model dengan distribusi bobot yang sehat dan menyebar (seperti ReLU) menunjukkan kemampuan belajar yang baik, dengan bukti berupa training dan validation loss yang menurun. Sementara itu, model dengan distribusi yang sangat sempit (seperti linear dan sigmoid) menunjukkan keterbatasan dalam belajar pola kompleks, yang berdampak pada tingginya training loss dan validation loss.

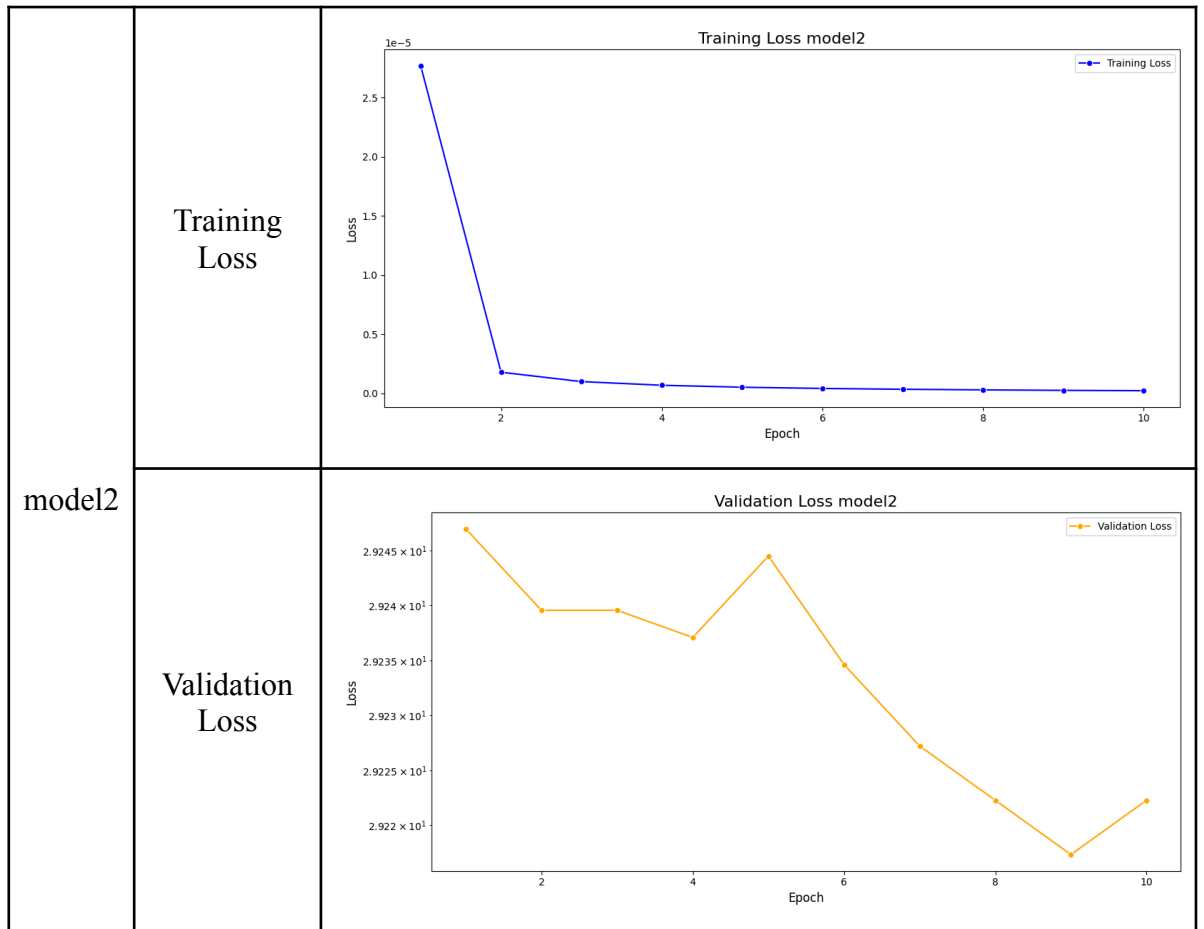
2.2.3 Pengaruh learning rate

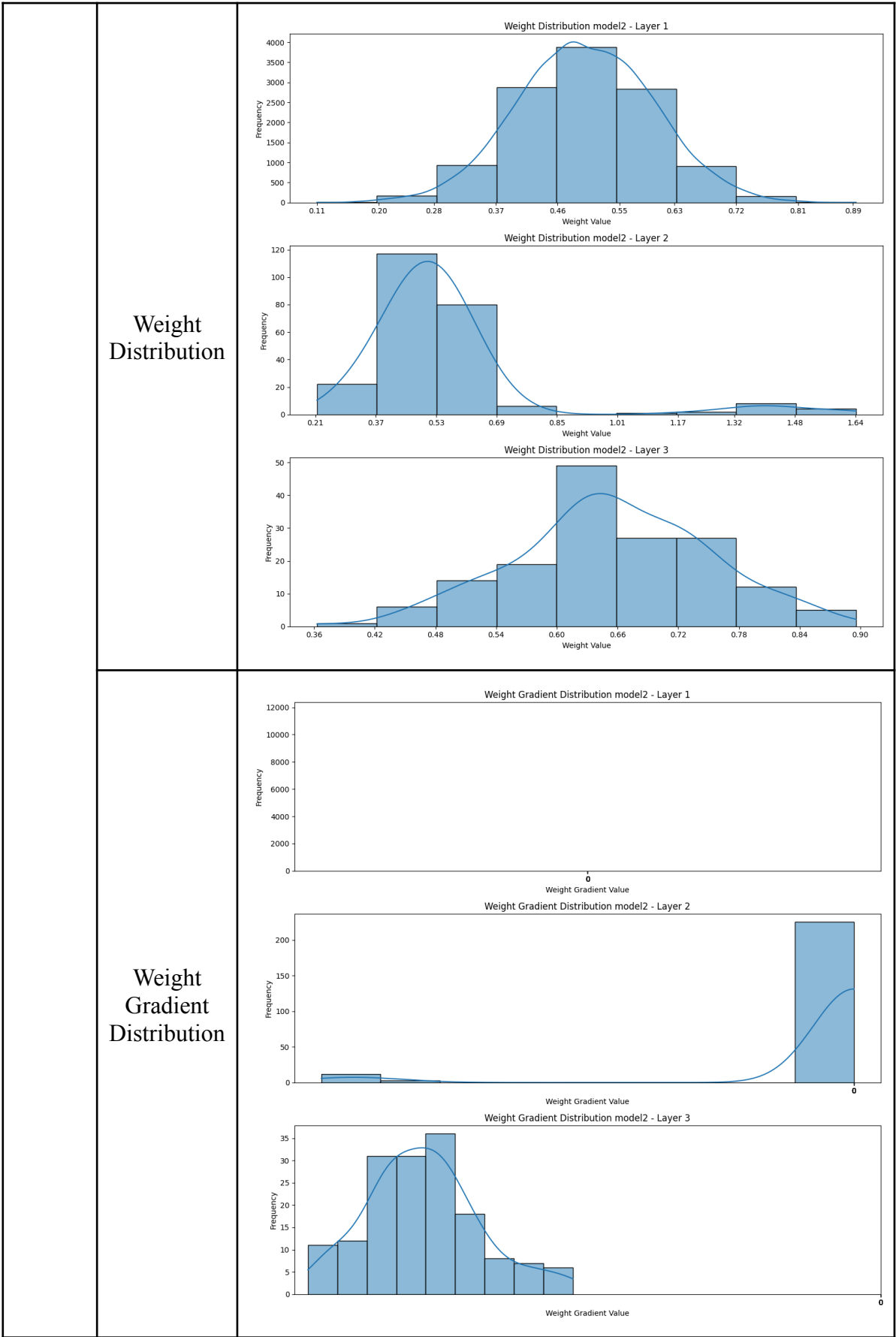
Dilakukan percobaan dengan 2 hidden layer dengan fungsi aktivasi relu dan 1 output layer dengan fungsi aktivasi sigmoid.

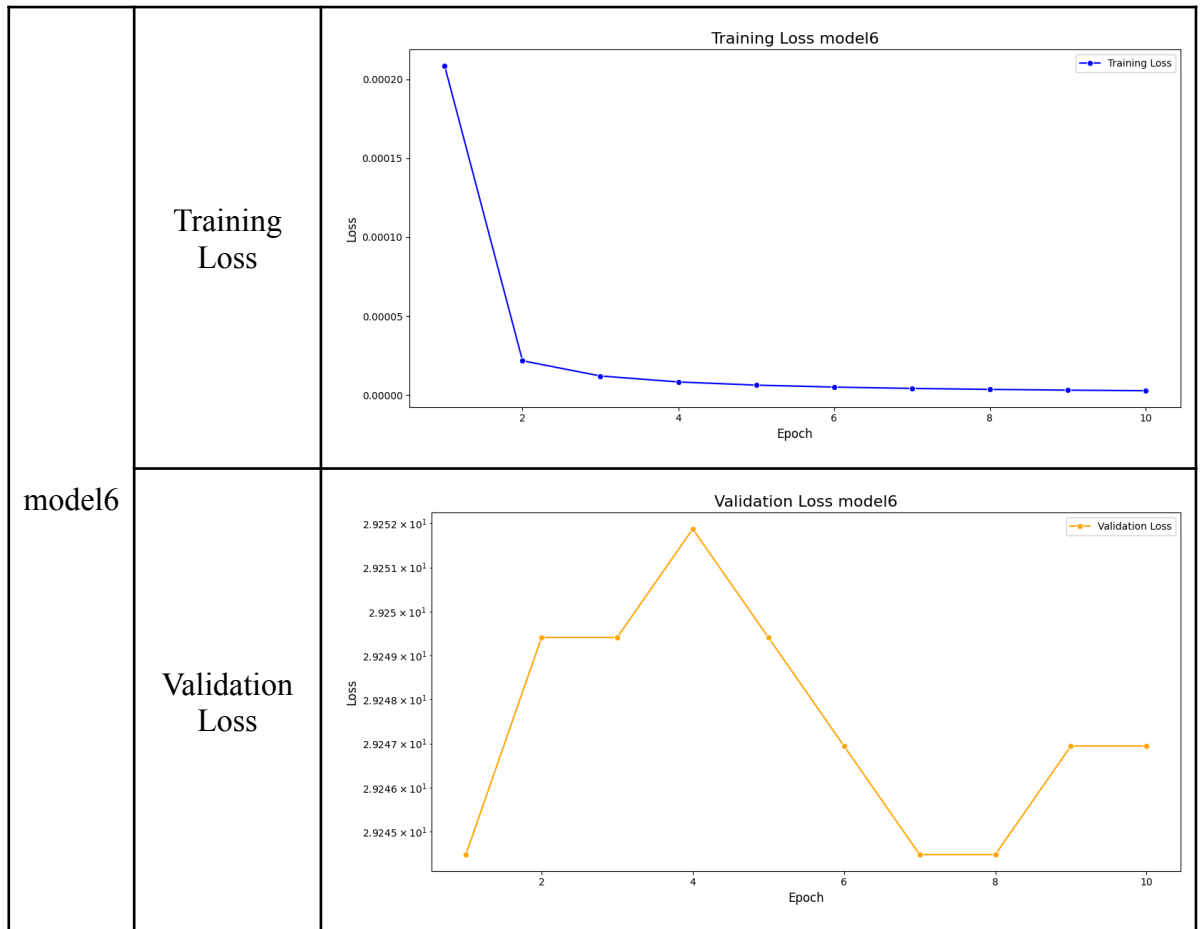
Model	Learning Rate	Score Prediksi
model2	0.1	0.1539
model6	0.01	0.1532
model7	0.001	0.1532
model8	0.0001	0.1531

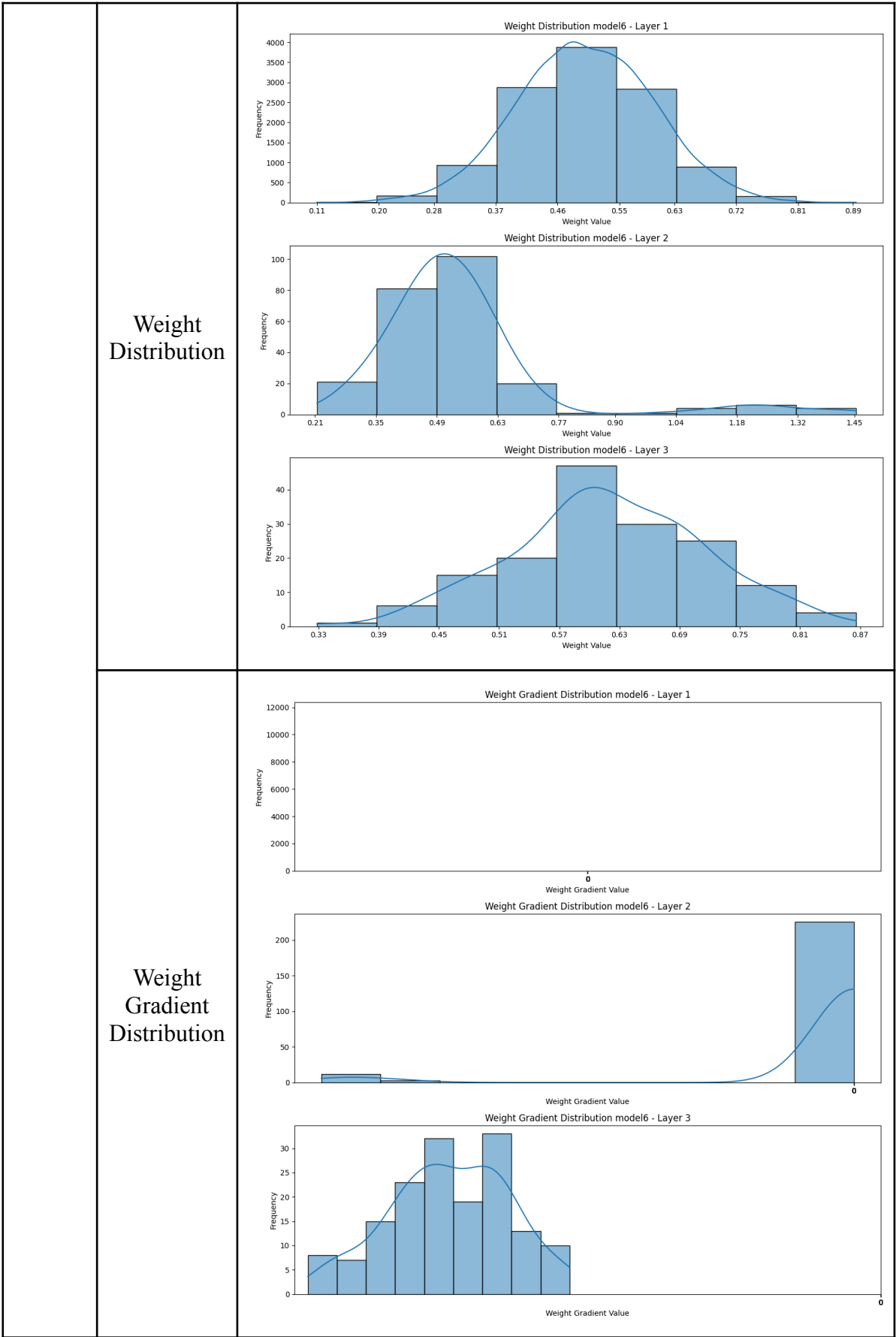
Tabel 2.2.3.1 Tabel Learning Rate

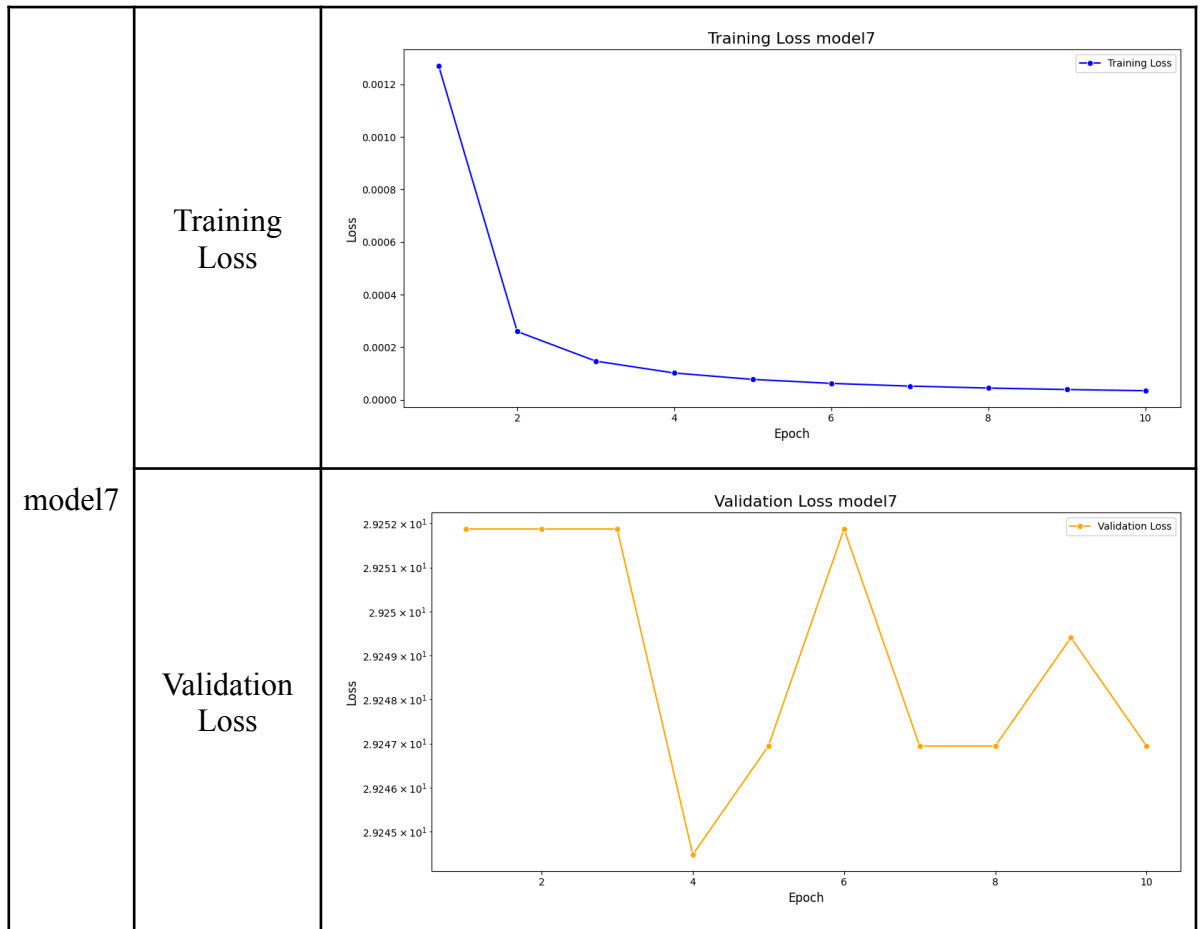
Model	Visualization
-------	---------------

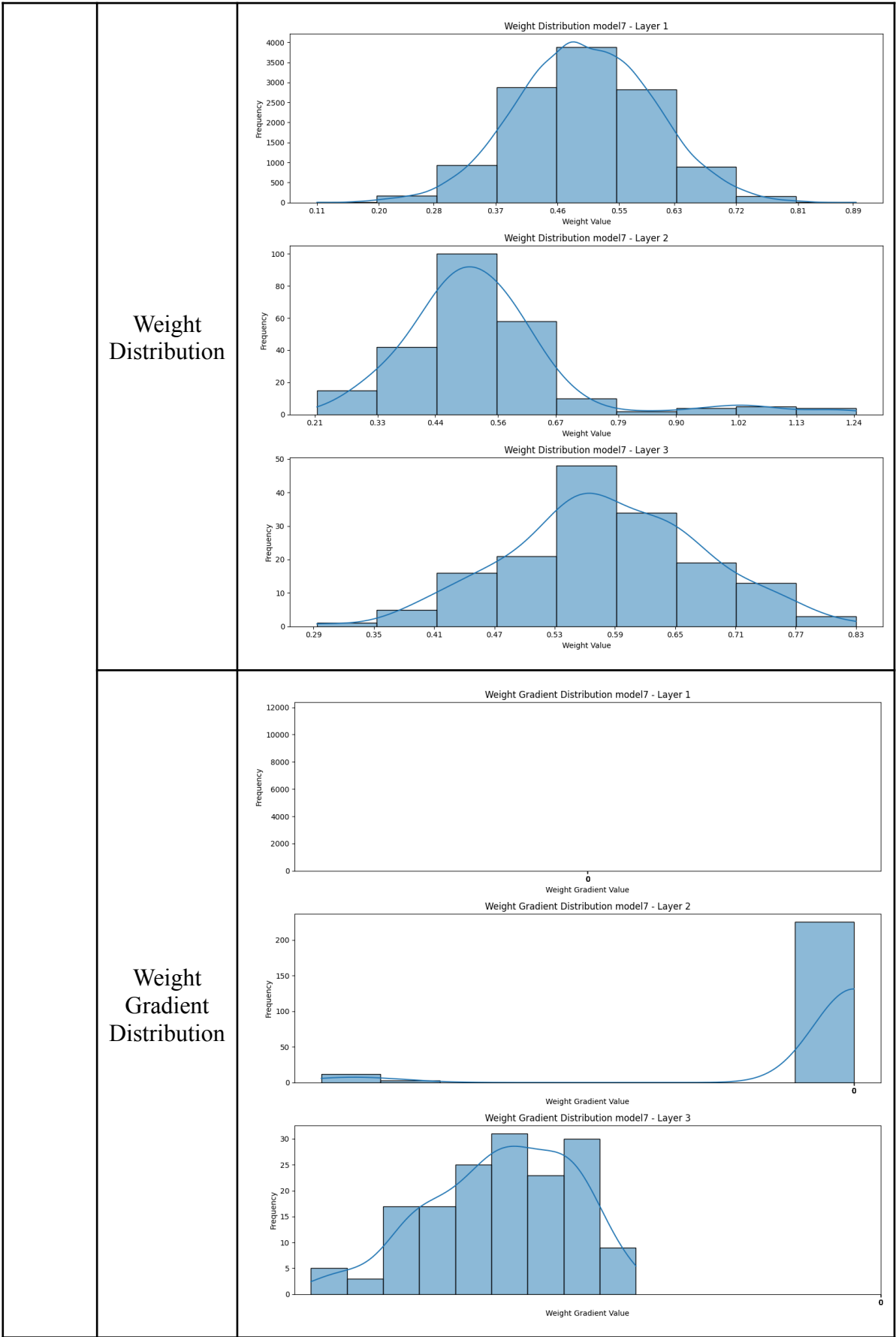


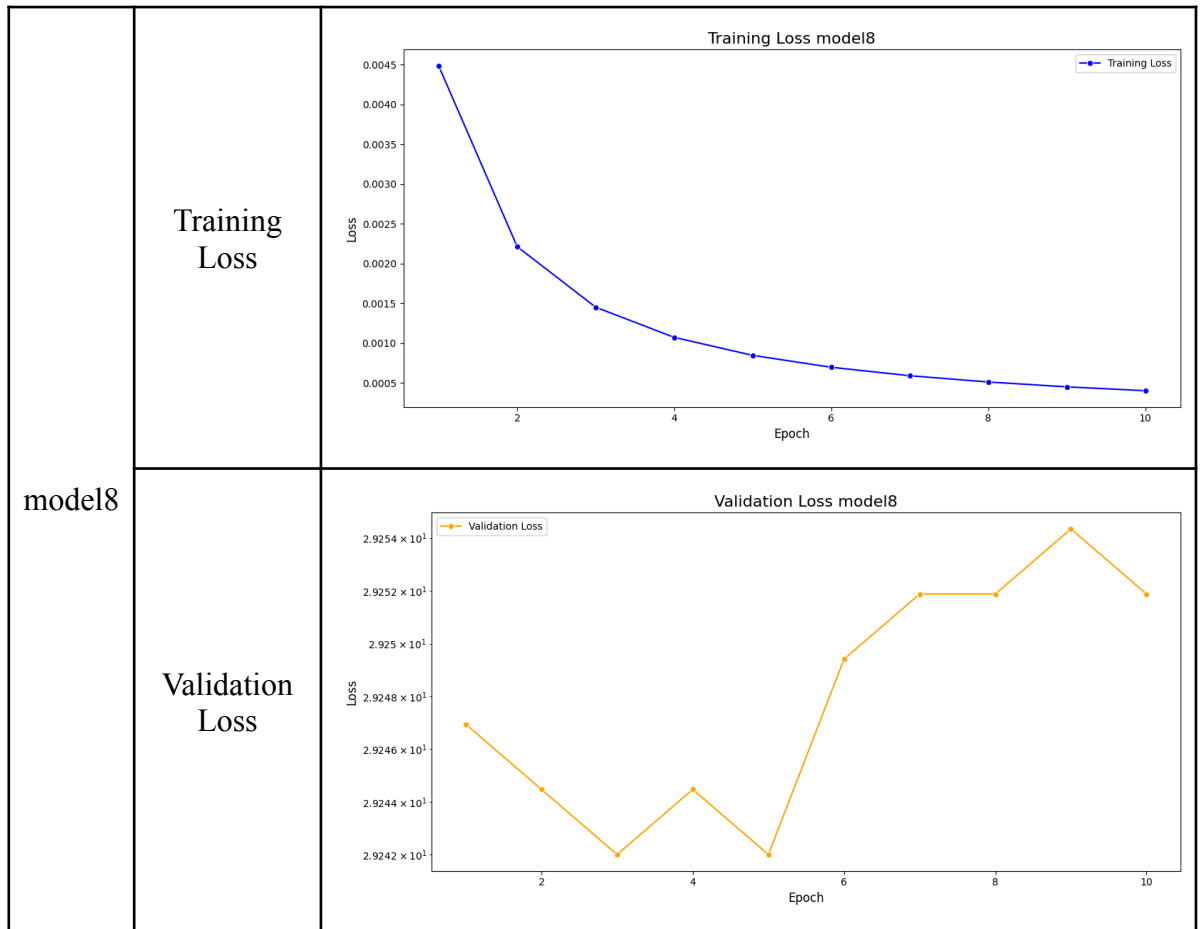


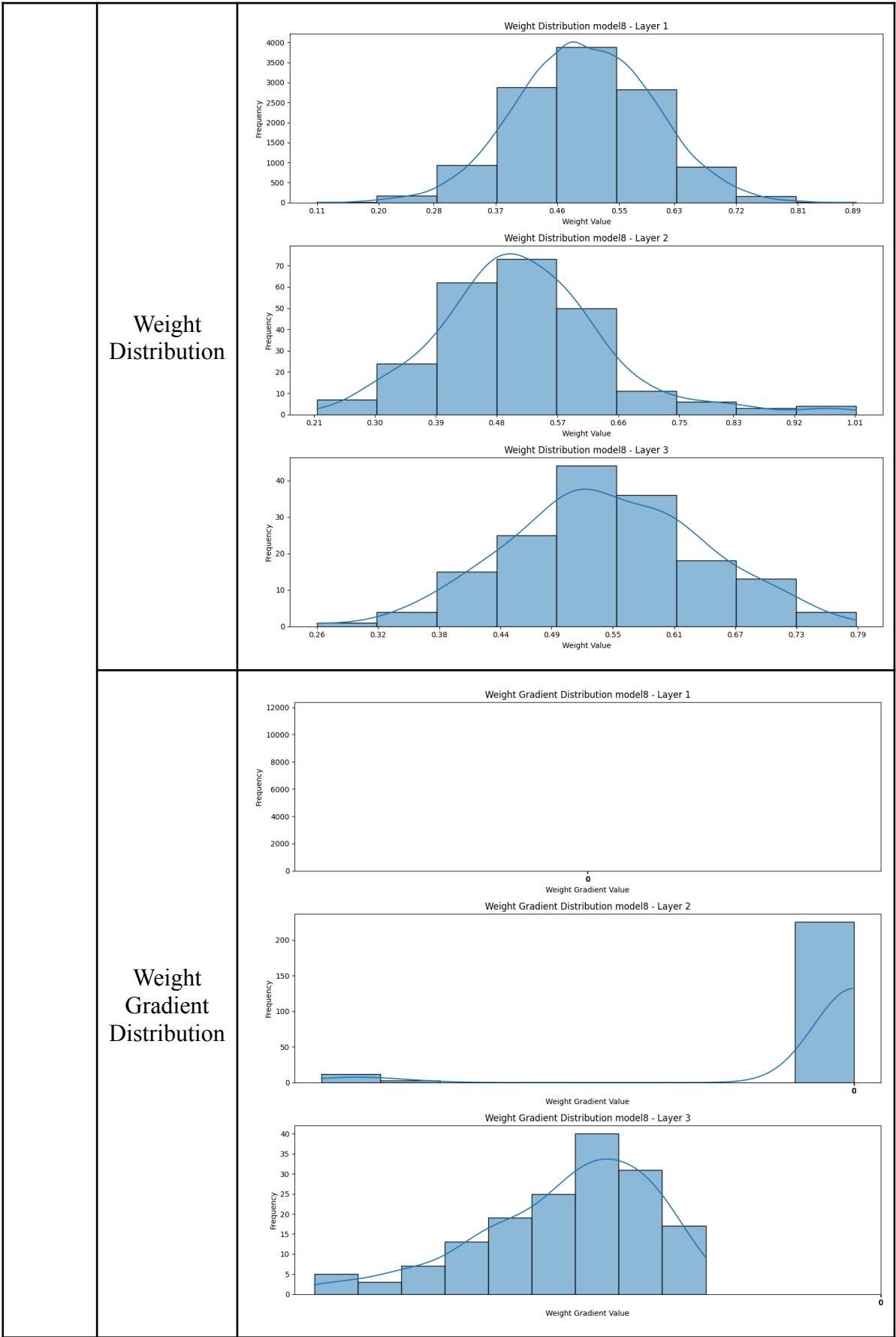












Tabel 2.2.3.2 Tabel Perbandingan Visualisasi Perbandingan Learning Rate

Dari hasil percobaan didapatkan bahwa semakin besar learning rate maka skor prediksi akan semakin besar, penurunan training loss pun semakin cepat. Namun, learning rate yang terlalu besar dapat mengakibatkan stabilitas training menjadi rendah dan tidak konvergen ke solusi optimal. **Grafik validation loss untuk model 2, model6, dan model7 cenderung turun, sedangkan untuk model8 mulai kembali naik mulai dari epoch 4.**

2.2.4 Pengaruh inisialisasi bobot

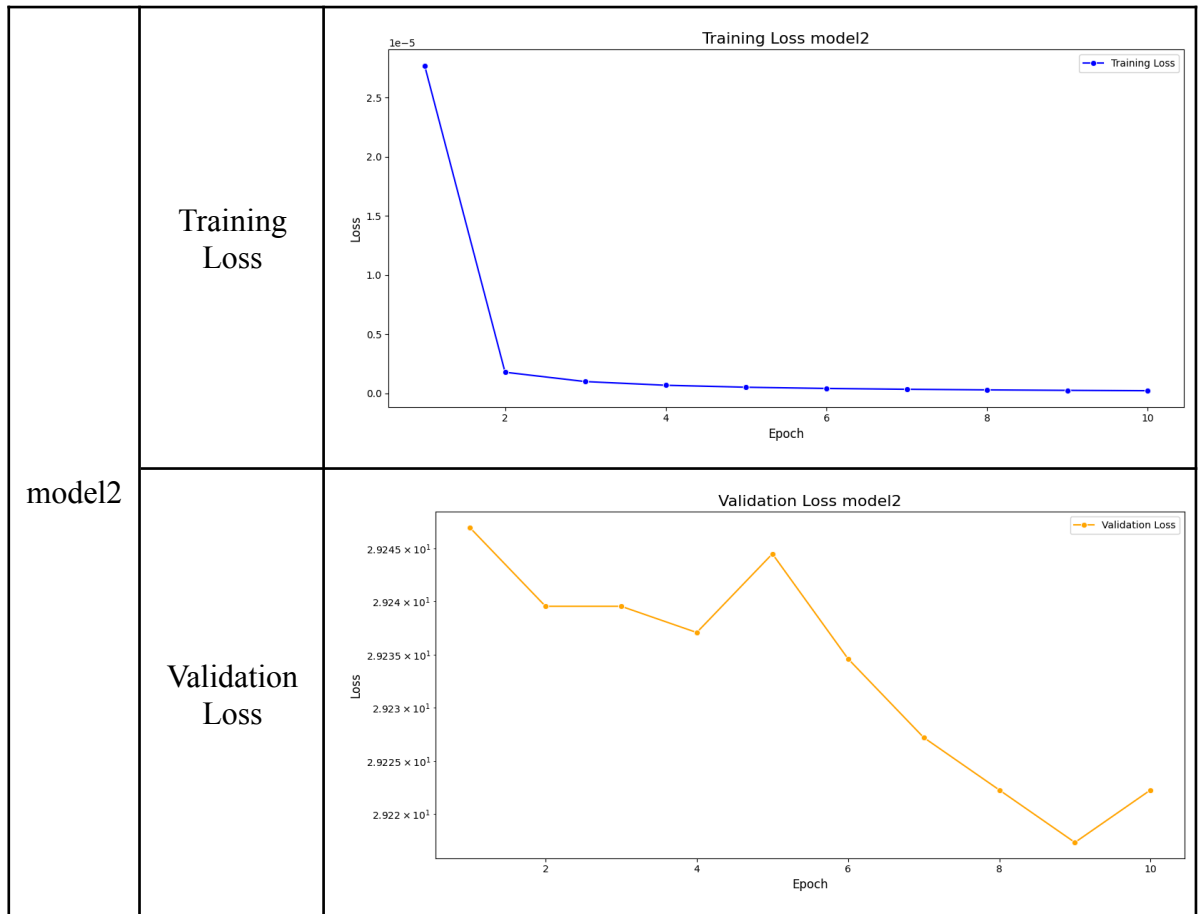
Dilakukan percobaan untuk membandingkan dampak tiga metode inisialisasi bobot terhadap performa pelatihan jaringan saraf tiruan. Metode yang digunakan meliputi:

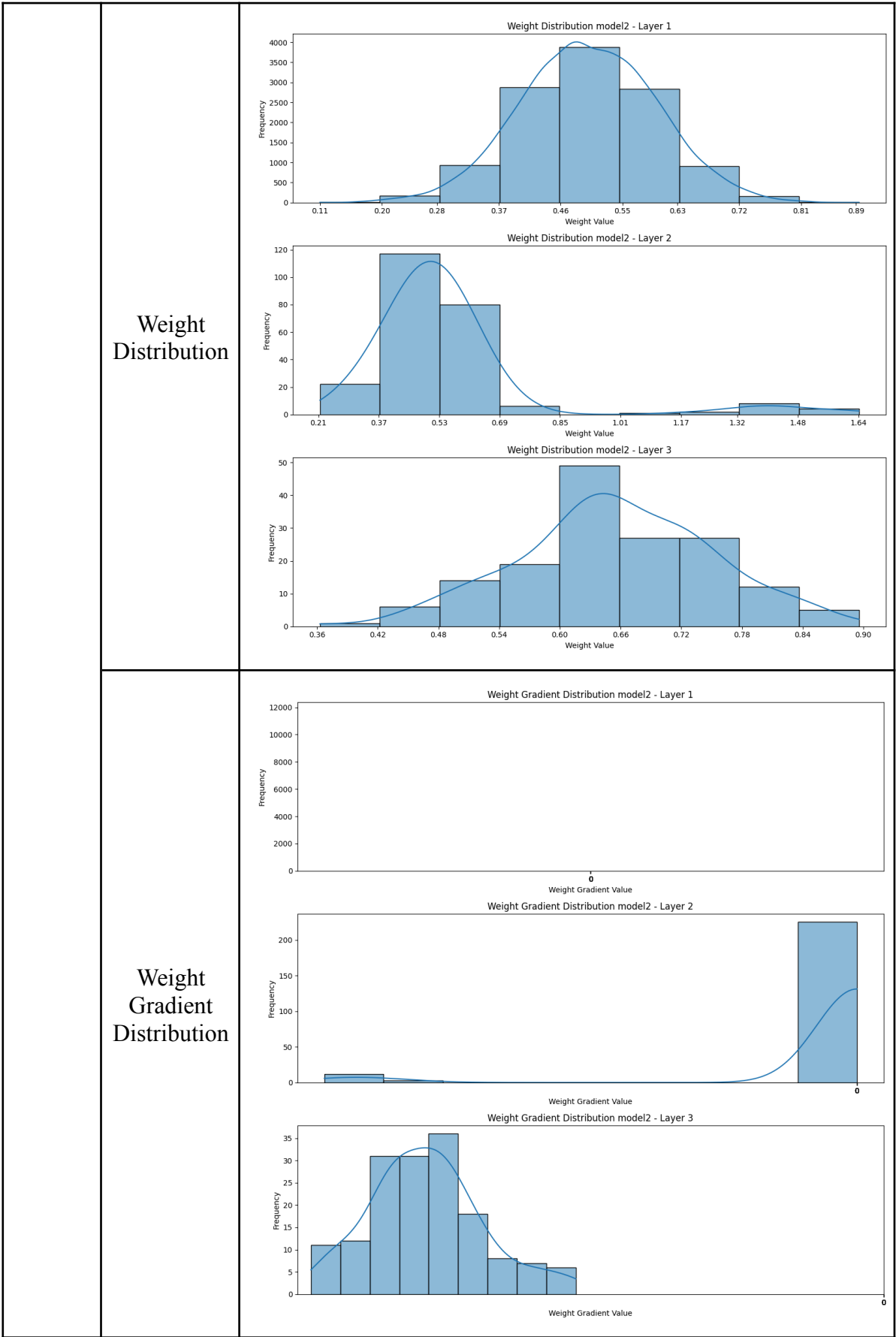
- 1) Normal: Inisialisasi bobot dari distribusi normal (gaussian)
- 2) Uniform: Inisialisasi bobot dari distribusi uniform
- 3) Zero: Semua bobot diinisialisasi dengan nilai nol

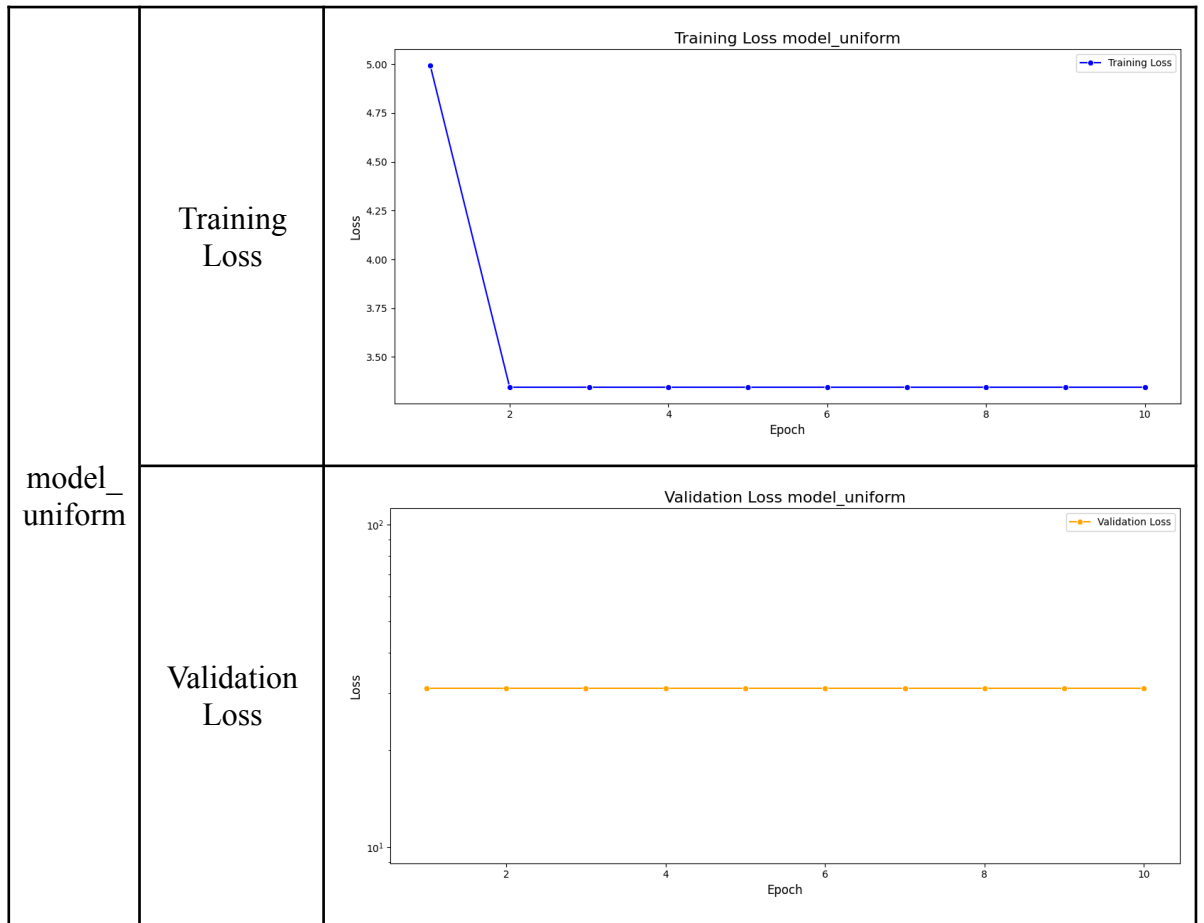
Model	Inisialisasi Bobot	Score Prediksi
model2	normal	1.539
model_uniform	uniform	1.001
model_zero	zero	0.1001

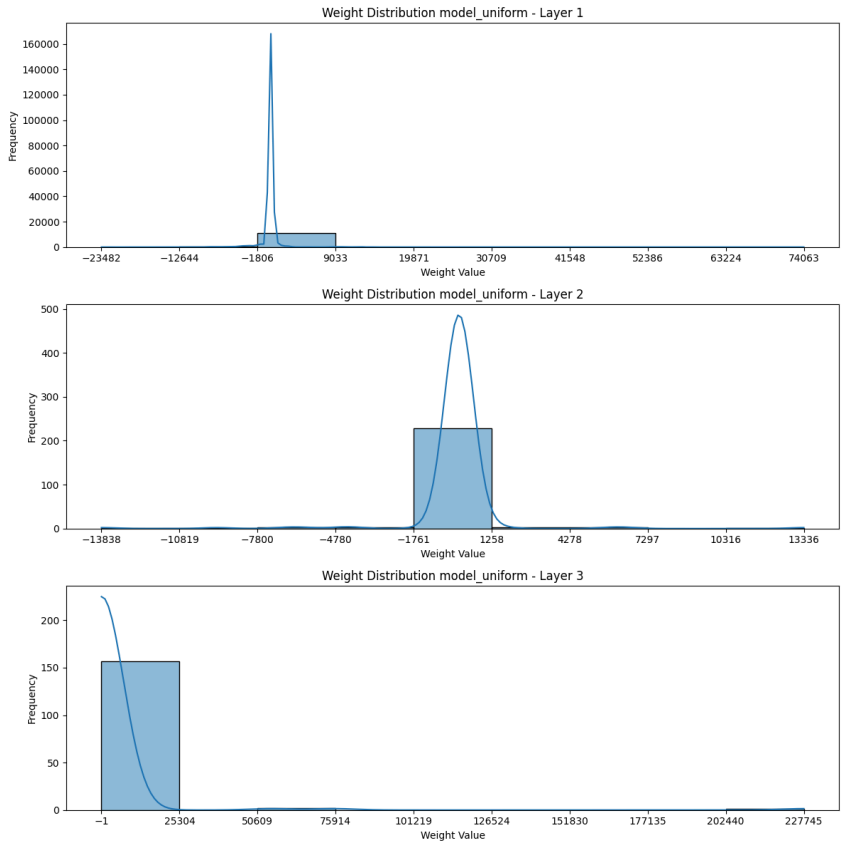
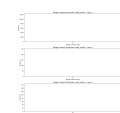
Tabel 2.2.4.1 Tabel Inisialisasi Bobot

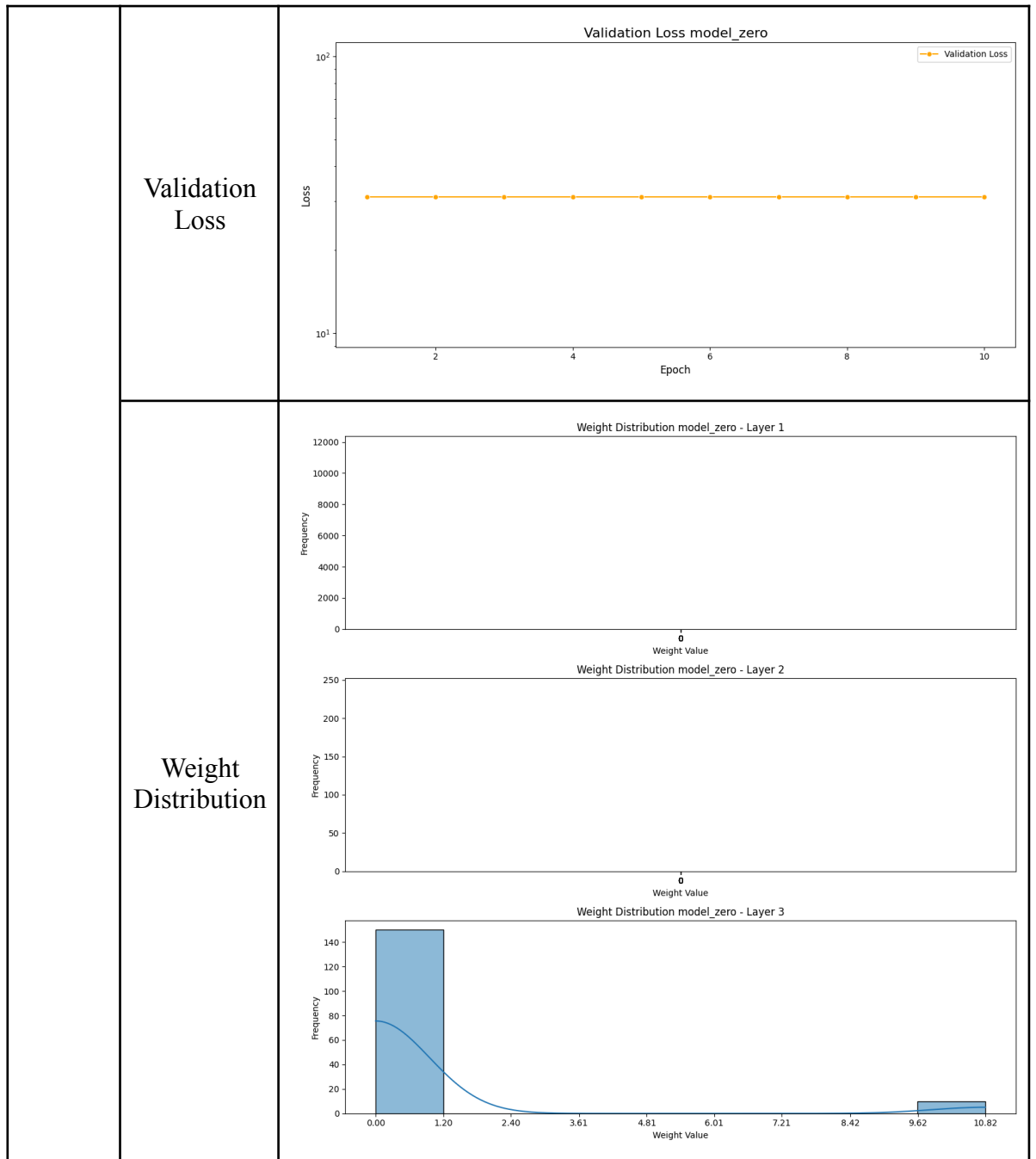
Model	Visualization
-------	---------------

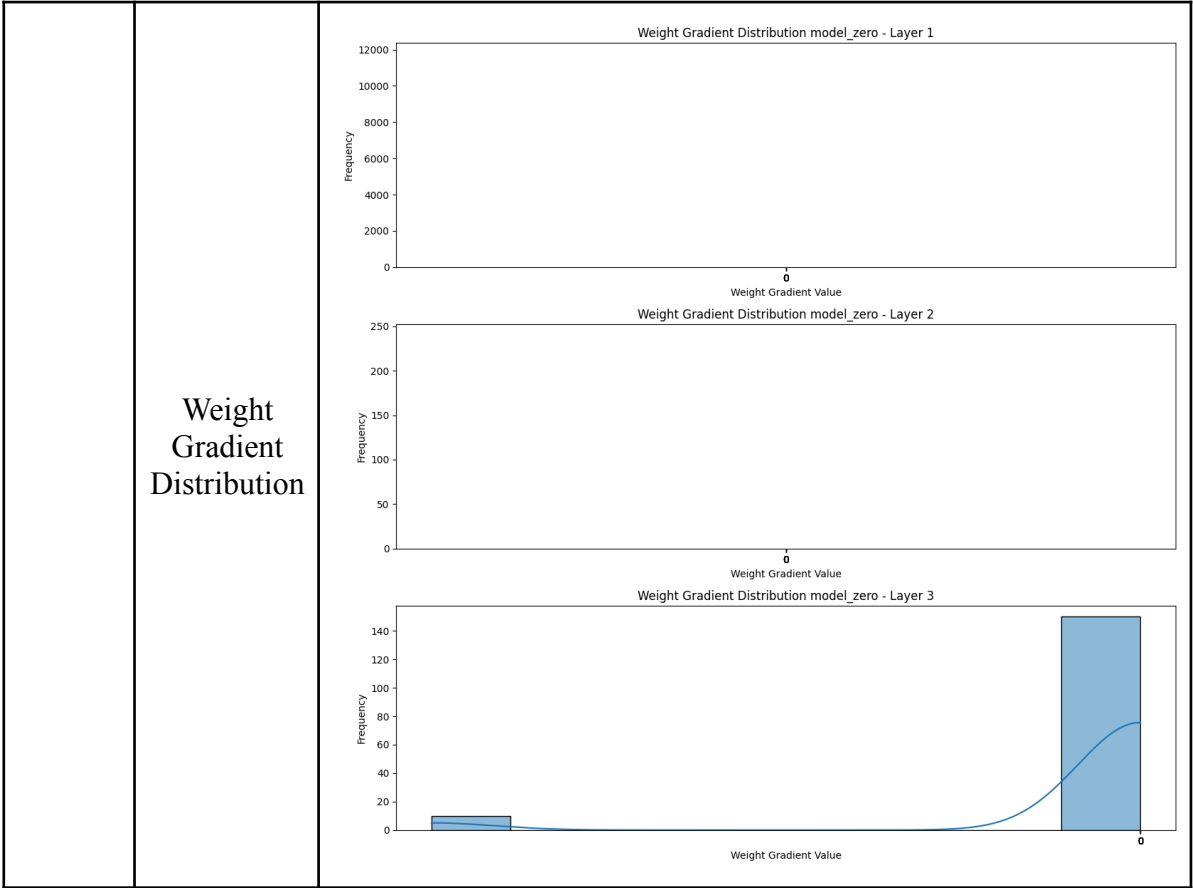






	Weight Distribution	 Weight Distribution model_uniform - Layer 1 Weight Distribution model_uniform - Layer 2 Weight Distribution model_uniform - Layer 3
	Weight Gradient Distribution	
model_zero	Training Loss	 Training Loss model_zero





Tabel 2.2.4.2 Tabel Perbandingan Visualisasi Perbandingan Inisialisasi Bobot

Pada eksperimen ini, dilakukan perbandingan tiga metode inisialisasi bobot, yaitu normal, uniform, dan zero. Hasil evaluasi menunjukkan bahwa model dengan inisialisasi bobot normal menghasilkan skor prediksi tertinggi, diikuti oleh uniform, sementara model dengan inisialisasi nol memiliki skor yang sangat rendah. Hal ini menunjukkan bahwa pemilihan metode inisialisasi berperan penting dalam efektivitas pelatihan jaringan.

Distribusi bobot dan gradien memberikan gambaran yang jelas mengenai perbedaan performa antar metode. Pada model dengan inisialisasi normal, training loss dan validation loss mengalami penurunan yang stabil dan signifikan. Distribusi bobotnya terlihat menyebar dengan baik dan simetris, memungkinkan model mengeksplorasi ruang parameter secara optimal. Gradien bobot yang terdistribusi merata mendukung proses pembaruan bobot secara efisien selama training.

Sementara itu, model dengan inisialisasi uniform masih menunjukkan penurunan pada training loss dan validation loss, meskipun tidak secepat dan sebesar model normal. Distribusi bobotnya lebih sempit, yang dapat membatasi variasi awal dan memperlambat proses pembelajaran. Gradien yang dihasilkan tetap stabil, meskipun tidak sebanyak pada model normal.

Berbeda dengan dua metode sebelumnya, model yang diinisialisasi dengan bobot nol mengalami kegagalan dalam belajar. Baik training loss maupun validation loss tidak menunjukkan penurunan yang berarti. Distribusi bobot tetap stagnan di nol, yang berarti tidak ada pembaruan parameter yang terjadi. Hal ini disebabkan oleh simetri dalam jaringan; karena semua neuron memulai dengan nilai bobot yang sama, mereka memproduksi output yang identik dan menerima gradien yang sama pula, sehingga tidak ada diferensiasi dalam pembelajaran.

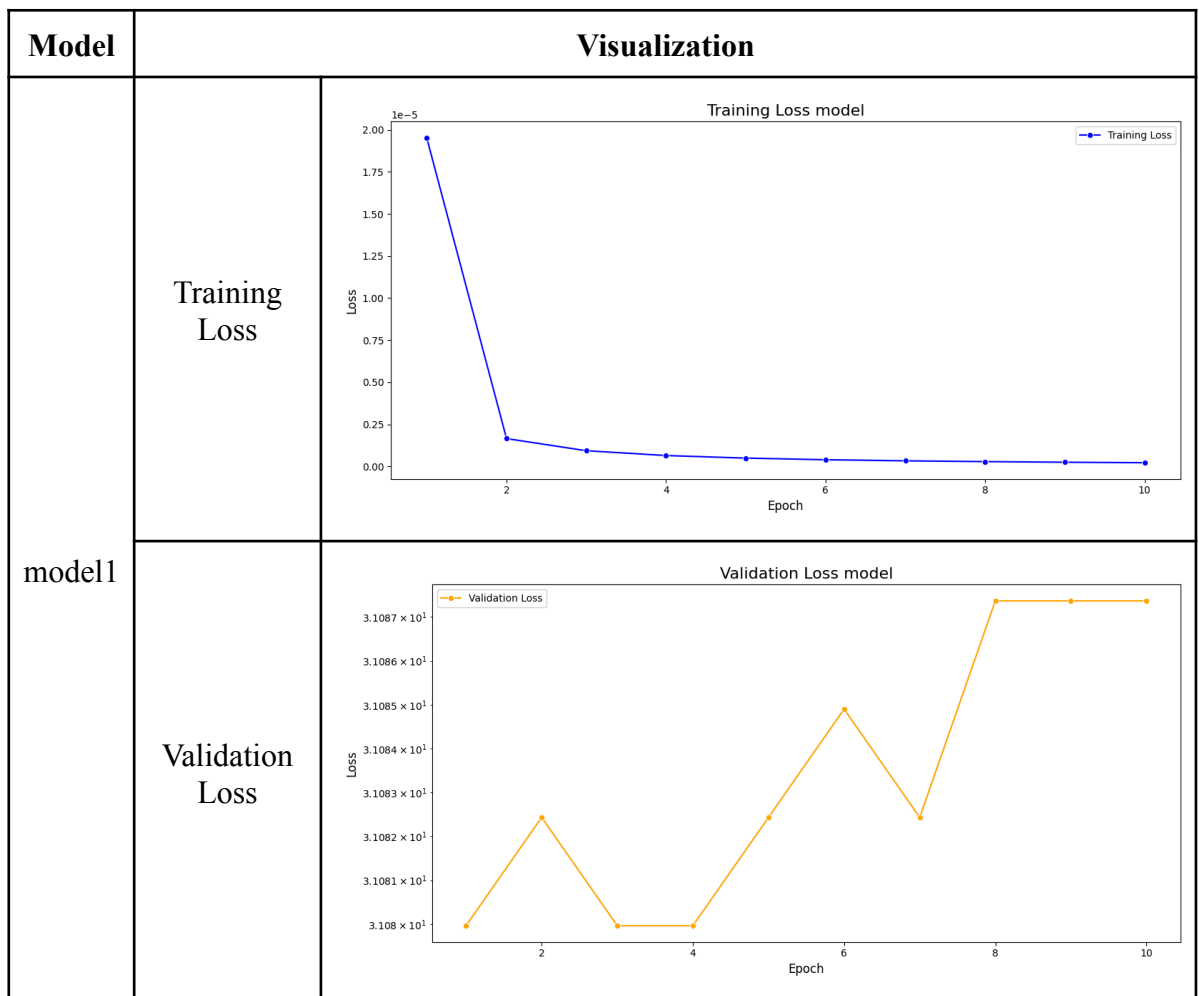
Secara keseluruhan, eksperimen ini menegaskan pentingnya pemilihan metode inisialisasi bobot yang tepat, di mana inisialisasi dengan distribusi normal memberikan performa terbaik dalam mendukung proses pelatihan yang efektif dan efisien.

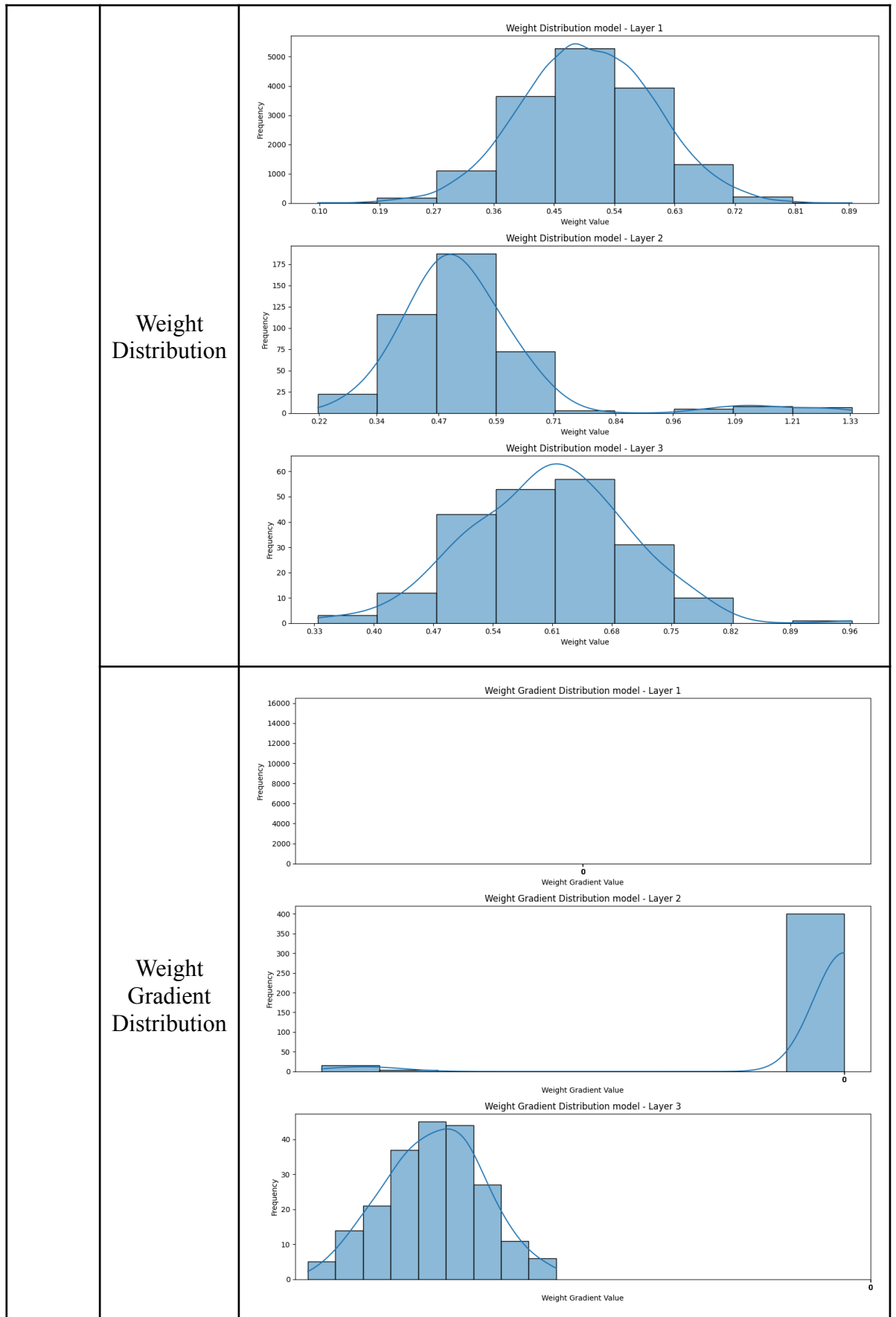
2.2.5 Pengaruh regularisasi

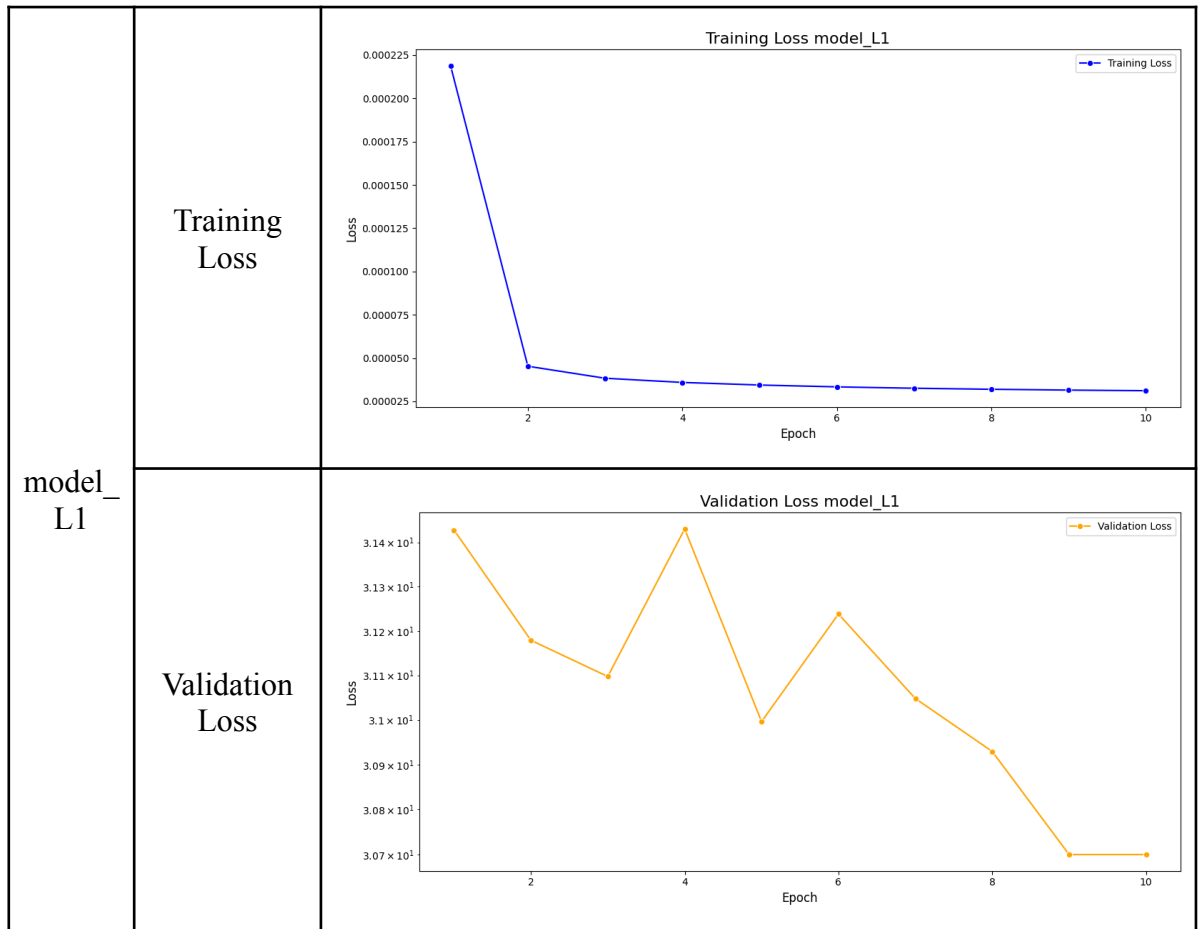
Hasil di bawah ini menunjukkan bahwa penambahan regularisasi, khususnya L1, sedikit meningkatkan akurasi model dibandingkan tanpa regularisasi. Regularisasi L1 mendorong sparsity pada bobot, yang bisa membantu dalam menghindari overfitting serta menghasilkan model yang lebih sederhana dan general.

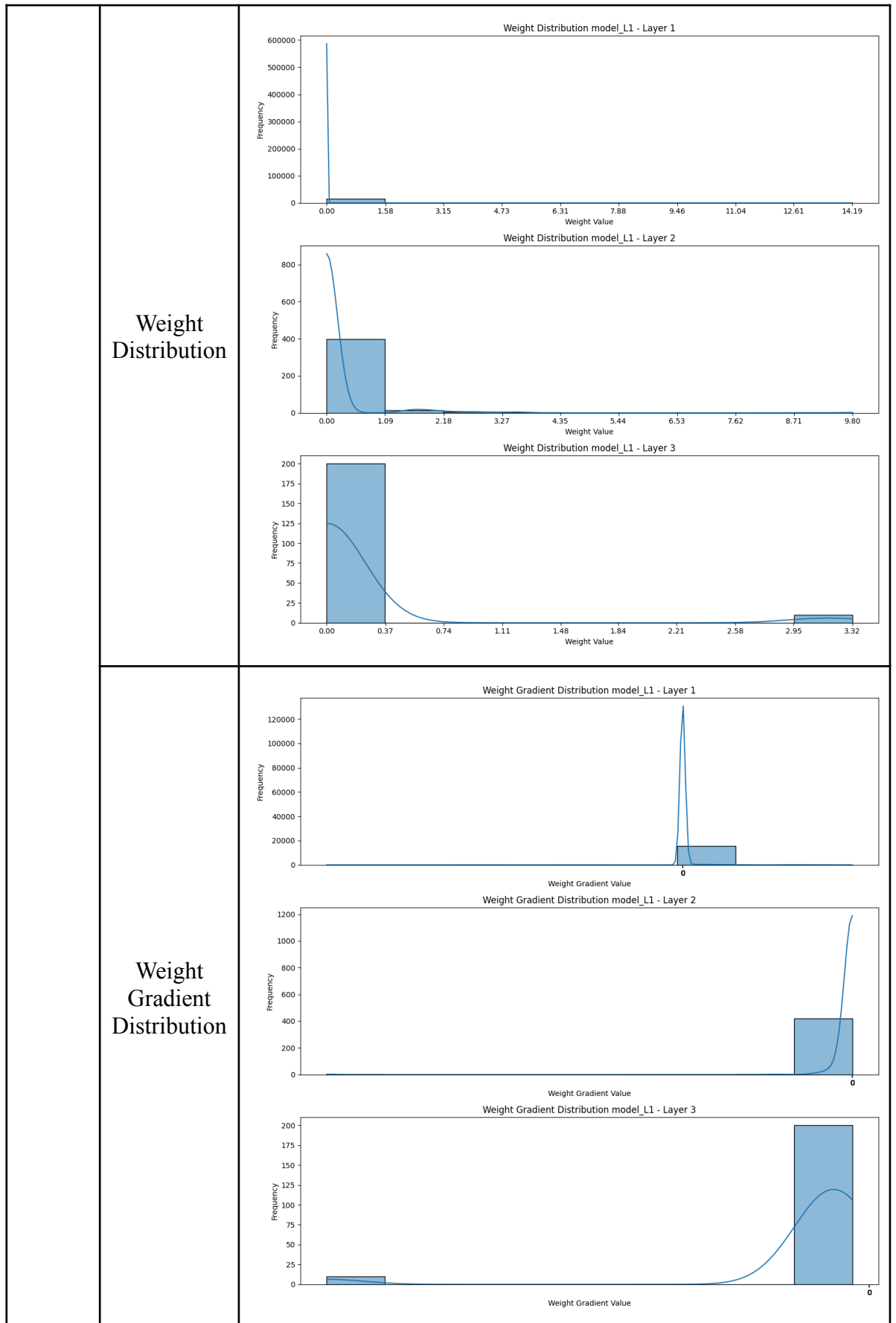
Model	Regularisasi	Score Prediksi
model1	-	0.0999
model_L1	L1 (lambda = 0.001)	0.1111
model_L2	L2 (lambda = 0.001)	0.0973

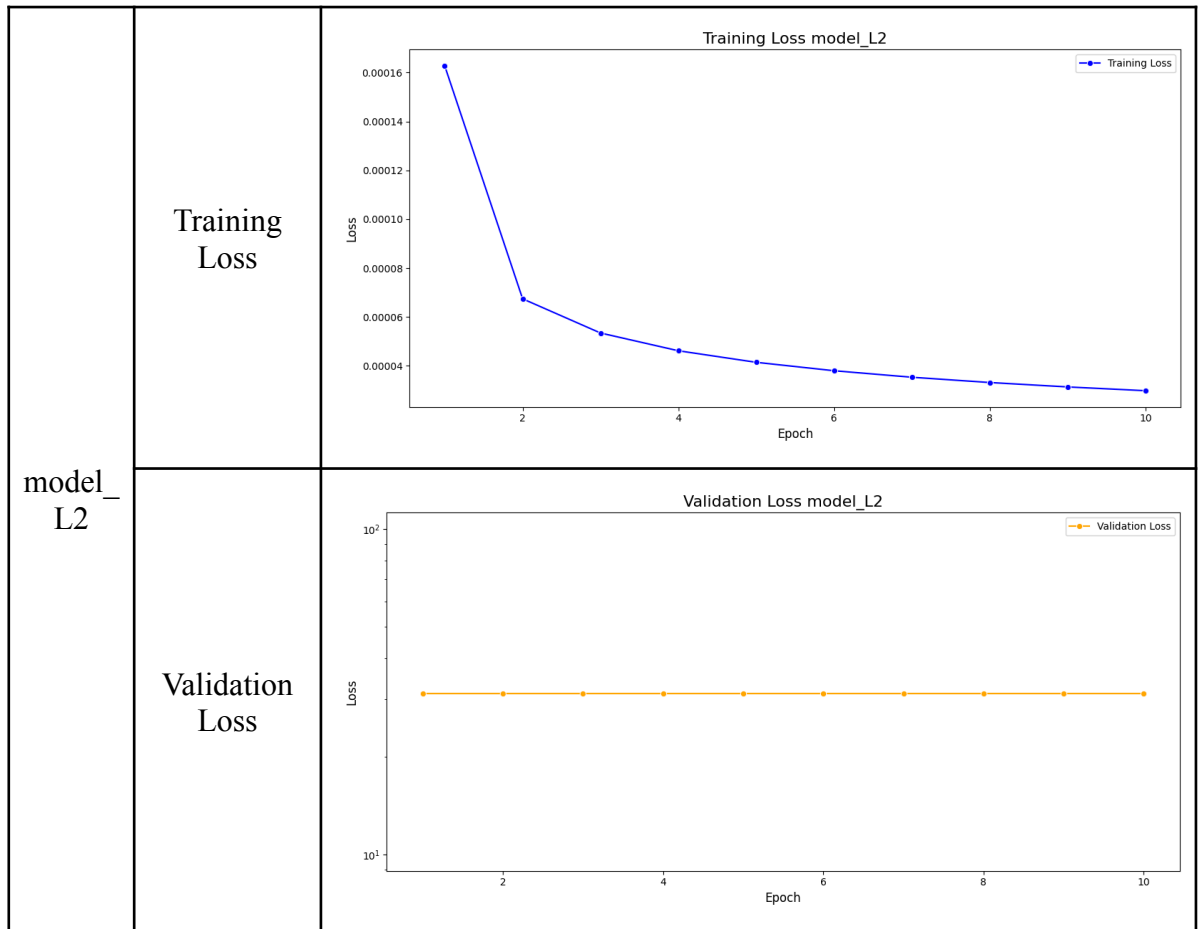
Tabel 2.2.5.1 Tabel Regularisasi

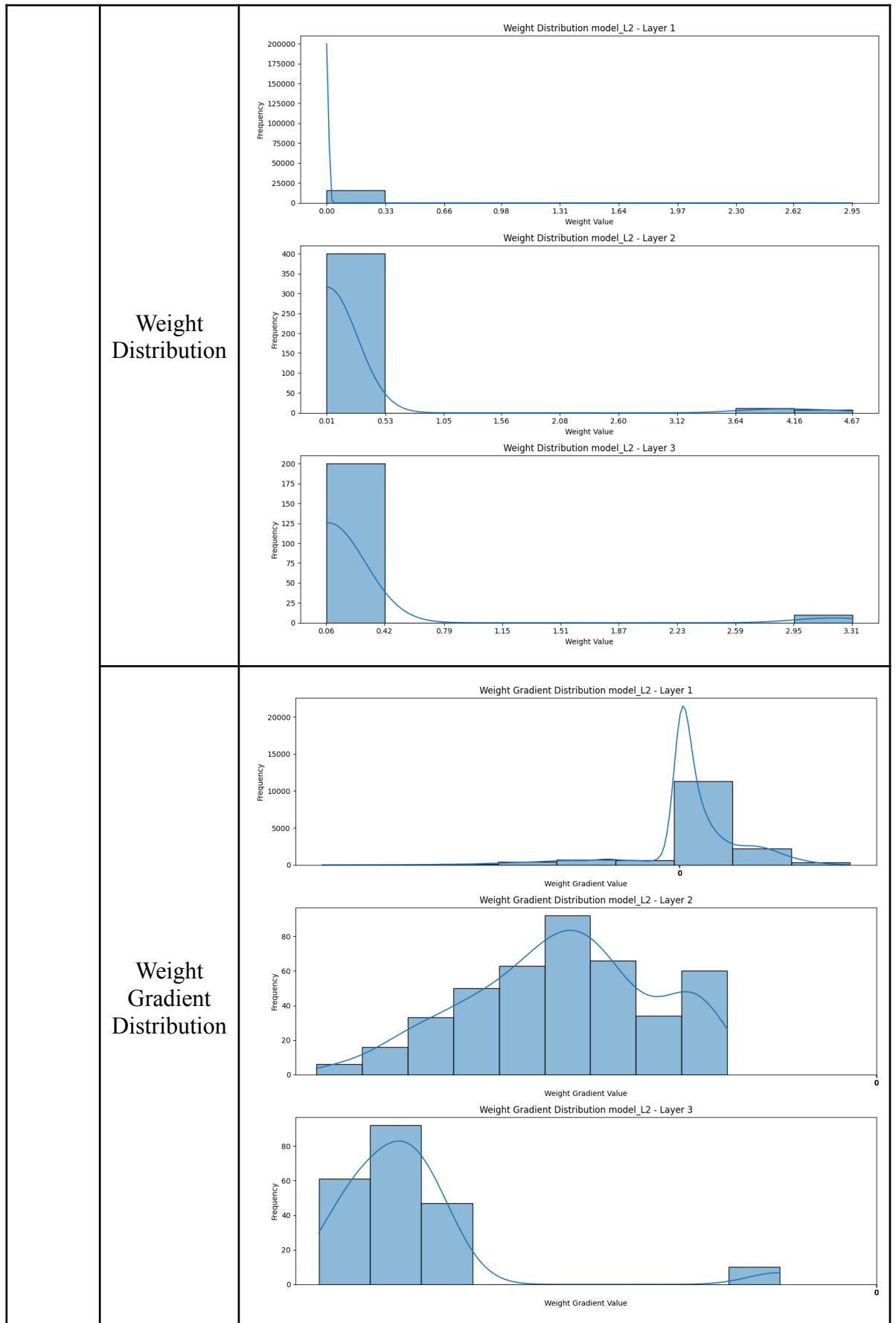












Tabel 2.2.5.2 Tabel Perbandingan Visualisasi Regularisasi

Penambahan regularisasi memberikan pengaruh positif dalam menstabilkan performa model, terutama regularisasi L1 yang menghasilkan skor prediksi tertinggi di antara ketiga model. Regularisasi juga terbukti efektif dalam mengontrol distribusi bobot, mencegah bobot menjadi terlalu besar dan meminimalkan overfitting. Penggunaan regularisasi perlu disesuaikan dengan kompleksitas data dan model untuk mendapatkan performa terbaik.

2.2.6 Perbandingan dengan library sklearn

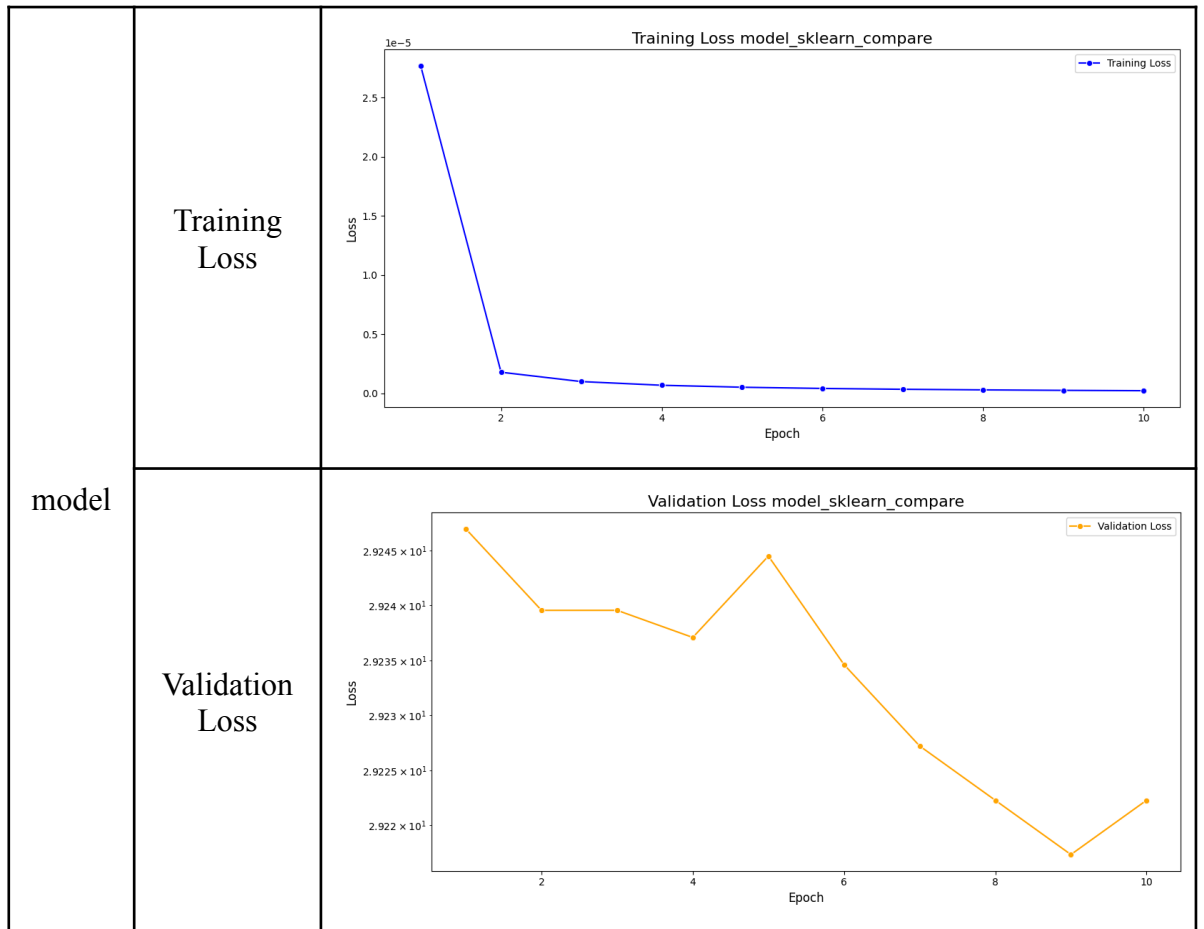
Dilakukan percobaan untuk membandingkan performa model Feed Forward Neural Network (FFNN) hasil implementasi sendiri dengan model MLPClassifier yang disediakan oleh library scikit-learn. Tujuannya adalah untuk mengevaluasi apakah model yang diimplementasikan dari nol memiliki performa yang sebanding dengan model standar dari library pembelajaran mesin yang umum digunakan.

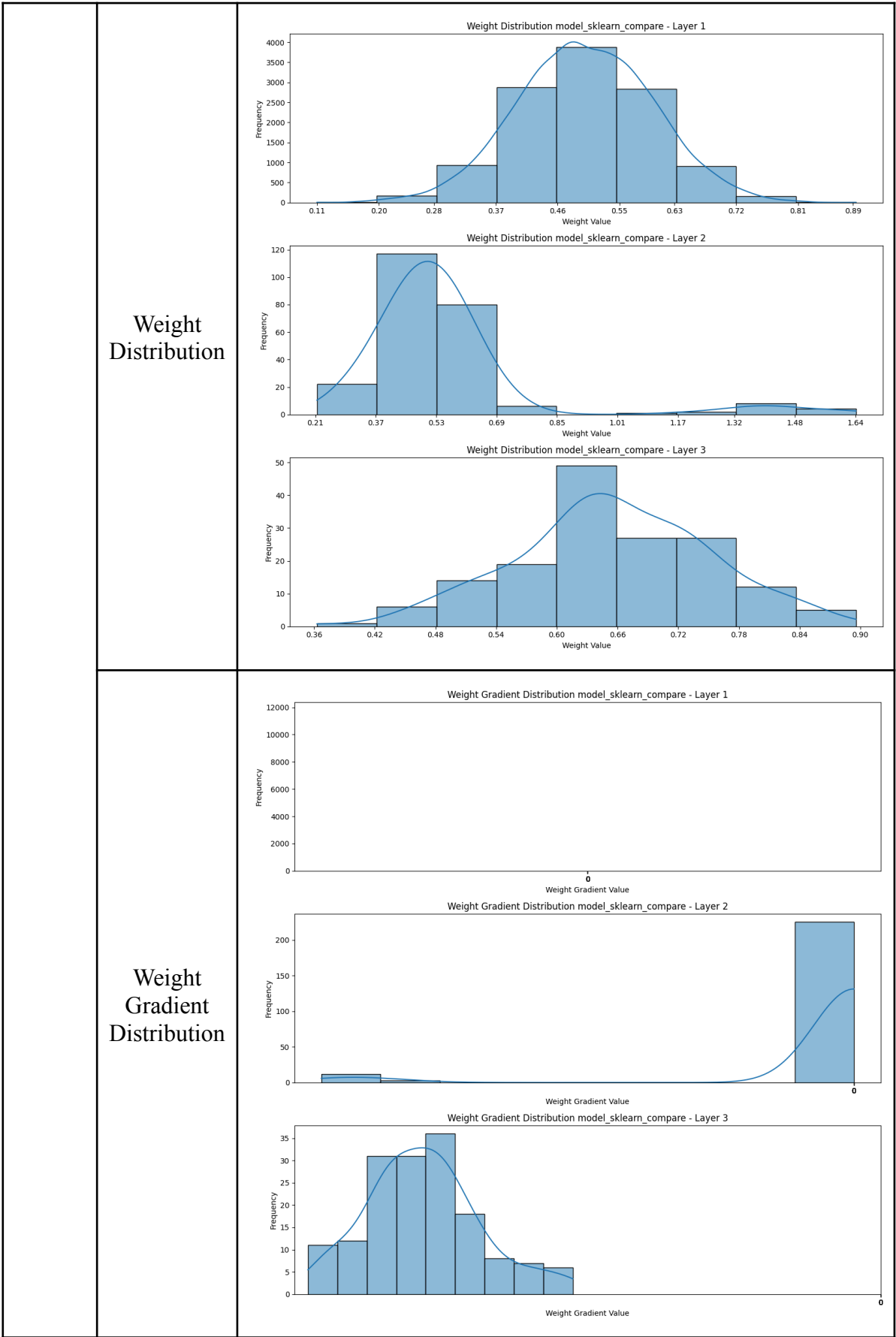
Berdasarkan hasil pada Tabel 2.2.6.1, model FFNN hasil implementasi sendiri berhasil mencapai skor prediksi sebesar 0.1539, sedangkan model dari sklearn hanya memperoleh skor 0.1001. Hal ini menunjukkan bahwa meskipun dibangun secara manual, model FFNN mampu memberikan hasil yang lebih baik daripada model MLPClassifier dari scikit-learn, setidaknya dalam konteks pengaturan parameter dan arsitektur yang digunakan dalam percobaan ini.

Model	Keterangan	Score Prediksi
model_sklearn_compare	FFNN (implementasi)	0.1539
model_sklearn	MLPClassifier (Sklearn)	0.1001

Tabel 2.2.6.1 Tabel Perbandingan Sklearn

Model	Visualization
-------	---------------





model_ sklearn	-	-
	-	-
	-	-
	-	-

Tabel 2.2.6.2 Tabel Perbandingan Visualisasi dengan Sklearn

Visualisasi pada Tabel 2.2.6.2 semakin memperkuat hasil tersebut. Model FFNN menunjukkan penurunan pada training loss dan validation loss, serta memiliki distribusi bobot dan gradien yang menggambarkan proses pembelajaran yang aktif dan seimbang. Sebaliknya, model sklearn tidak disertai visualisasi loss maupun distribusi bobot dan gradien dalam laporan, sehingga tidak dapat dievaluasi secara visual. Namun, skor prediksi yang lebih rendah mengindikasikan bahwa model sklearn mungkin kurang optimal pada dataset ini, atau menggunakan parameter default yang tidak sepenuhnya sesuai dengan kebutuhan model.

Dengan demikian, dapat disimpulkan bahwa implementasi FFNN dari nol dapat memberikan hasil yang sangat kompetitif, bahkan melampaui performa model standar dari sklearn, selama parameter, arsitektur, dan teknik pelatihan yang digunakan disesuaikan dengan baik terhadap dataset yang digunakan. Selain menjadi pembuktian atas keberhasilan implementasi, hal ini juga menunjukkan pentingnya pemahaman mendalam terhadap proses training model yang sering tersembunyi di balik library siap pakai.

3. Kesimpulan dan Saran

3.1 Kesimpulan

Secara umum, tugas ini menuntut pemahaman tentang metode klasifikasi dalam machine learning dengan menerapkan berbagai teknik eksplorasi data, preprocessing, pemilihan model, pelatihan, evaluasi, dan interpretasi hasil.

3.2 Saran

Untuk menyelesaikan tugas besar ini dengan optimal, sangat penting untuk memahami dengan baik tujuan utama yang ingin dicapai, yaitu melakukan klasifikasi customer segmentation menggunakan metode pembelajaran mesin.

Pemahaman yang mendalam terhadap dataset yang digunakan menjadi langkah awal yang krusial. Dalam tahap awal, eksplorasi data harus dilakukan untuk mengidentifikasi pola, distribusi, serta potensi adanya missing values atau outlier yang dapat mempengaruhi hasil klasifikasi. Proses ini juga sebaiknya dilengkapi dengan visualisasi data agar lebih mudah dalam memahami karakteristik dari setiap fitur yang tersedia.

4. Pembagian tugas tiap anggota kelompok

Nama (NIM)	Kontribusi
Aurelius Justin Philo Fanjaya (13522020)	FFNN class, forward & back propagation, training, testing, autodiff
Amalia Putri (13522042)	Activation Function, Loss Function, melengkapi dokumen 50%
Fedrianz Dharma (13522090)	Visualisasi Weight Distribution, Visualisasi Weight Gradient Distribution, Laporan, Debug dan Fixing

Tabel 4.1 Tabel Pembagian Tugas Kelompok

5. Referensi

-  The spelled-out intro to neural networks and backpropagation: building microg...
- <https://www.jasonosajima.com/forwardprop>
- <https://www.jasonosajima.com/backprop>
- <https://numpy.org/doc/2.2/>
- https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html
- [https://math.libretexts.org/Bookshelves/Calculus/Calculus_\(OpenStax\)/14%3A_Differentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariable_Functions](https://math.libretexts.org/Bookshelves/Calculus/Calculus_(OpenStax)/14%3A_Differentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariable_Functions)
- <https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/>
- <https://douglasorr.github.io/2021-11-autodiff/article.html>
- https://www.cs.toronto.edu/~rgrosse/courses/csc2541_2022/tutorials/tut01.pdf